SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

THANJAVUR | KUMBAKONAM | CHENNAI

# DATA SCIENCE USING PYTHON - MAT337

## LAB RECORD

**Submitted By**

**Student Name** :

**Register Number** :

**Programme** : M. Sc. Data Science - I

**Academic Year** : 2026–2027

**School** : School of Arts, Sciences, Humanities & Education

**Internal Faculty Signature** **External Faculty Signature**

# TABLE OF CONTENTS


| S.No. | Date | Name of the Experiment | Page No. |
|---|---|---|---|
| 1 | 17.07.2026 | Implementation of Basic Python Programs Using Conditional Statements and Loops | 3 |
| 2 | 24.07.2026 | String Manipulation and Pattern-Based Programming in Python | 11 |
| 3 | 31.07.2026 | List Processing and User-Defined Functions in Python | 20 |
| 4 | 07.08.2026 | Advanced List And String Problem Solving Techniques | 24 |
| 5 | 08.08.2026 | Python List Comprehensions | 29 |
| 6 | 14.08.2026 | Dictionary Operations, Frequency Analysis And Comprehensions | 33 |
| 7 | 21.08.2026 | Generator Expressions And Lazy Evaluation Techniques In Python | 38 |
| 8 | 04.09.2026 | Numerical Computing And Matrix Operations Using Numpy | 42 |
| 9 | 11.09.2026 | Advanced Numerical Simulation And Statistical Analysis Using Numpy And Generator Expressions | 48 |
| 10 | 25.09.2026 | Linear Algebra Computations Using Numpy Linalg Package | 55 |
| 11 | 09.10.2026 | Data Cleaning, Preprocessing And Visualization Pipeline Using Csv Datasets | 63 |
| 12 | 10.10.2026 | Data Visualization Using Line, Bar, Scatter And Pie Charts | 68 |
| 13 | 16.10.2026 | Probability Distributions And Visualization | 83 |

# EXPERIMENT 1: IMPLEMENTATION OF BASIC PYTHON PROGRAMS USING CONDITIONAL STATEMENTS AND LOOPS

**AIM:**

To write and execute basic Python programs covering mathematical operations, string manipulations, list processing, conditional statements, loops, and function definitions.

**PROCEDURE:**

- Start the Python interpreter or open Python IDLE
- Create a new Python program for each of the given experiments.
- Read the required input values from the user using the input() function.
- Perform mathematical operations such as addition, swapping numbers, finding the largest number, calculating sums, averages, and factorials.
- Apply string manipulation techniques to reverse strings, count digits, check palindromes, and count vowels.
- Use lists and dictionaries to process elements, remove duplicates, count even numbers, and calculate word frequencies.
- Implement conditional statements such as if, else, and for loops to check even or odd numbers, identify Armstrong numbers, generate Fibonacci numbers, and find prime numbers.
- Define and call user-defined functions to perform addition and find the maximum element in a list.
- Execute each program using suitable input values and verify the output.
- Record the program code and corresponding outputs in the lab record.

**PROGRAM 1: WRITE A PYTHON PROGRAM TO CALCULATE THE SUM OF FIRST N NATURAL NUMBERS**

```
n = int(input("Enter n: "))
total = n * (n + 1) // 2
print("Sum:", total)
```

**OUTPUT:**

Enter n: 10
Sum: 55

**PROGRAM 2: WRITE A PYTHON PROGRAM TO SWAP TWO NUMBERS WITHOUT USING A TEMPORARY VARIABLE**

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
a, b = b, a
print("a =", a, "b =", b)
```

**OUTPUT:**

Enter a: 10
Enter b: 24
a = 24 b = 10

**PROGRAM 3: WRITE A PYTHON PROGRAM TO FIND THE LARGEST OF THREE NUMBERS**

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
c = int(input("Enter c: "))
print("Largest:", max(a, b, c))
```

**OUTPUT:**

Enter a: 44
Enter b: 22
Enter c: 54
Largest: 54

**PROGRAM 4: WRITE A PYTHON PROGRAM TO REVERSE A STRING**

```
s = input("Enter a string: ")
print("Reversed string:", s[::-1])
```

**OUTPUT:**

Enter a string: HELLO WORLD
Reversed string: DLROW OLLEH

**PROGRAM 5: WRITE A PYTHON PROGRAM TO COUNT THE NUMBER OF DIGITS IN AN INTEGER**

```
n = int(input("Enter an integer: "))
print("Number of digits:", len(str(abs(n))))
```

**OUTPUT:**

Enter an integer: 344324
Number of digits: 6

**PROGRAM 6: WRITE A PYTHON PROGRAM TO CALCULATE THE SUM OF ELEMENTS IN A LIST**

```
lst = list(map(int, input("Enter list elements: ").split()))
print("Sum:", sum(lst))
```

**OUTPUT:**

Enter list elements: 12 32 53 55
Sum: 152

**PROGRAM 7: WRITE A PYTHON PROGRAM TO CALCULATE THE AVERAGE MARKS OF STUDENTS**

```
marks = list(map(float, input("Enter marks: ").split()))
average = sum(marks) / len(marks)
print("Average marks:", average)
```

**OUTPUT:**

Enter marks: 45 23 24 12 55
Average marks: 31.8

**PROGRAM 8: WRITE A PYTHON PROGRAM TO CHECK WHETHER A NUMBER IS EVEN OR ODD**

```
n = int(input("Enter an integer: "))
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

**OUTPUT:**

Enter an integer: 48
Even

**PROGRAM 9: WRITE A PYTHON PROGRAM TO CHECK WHETHER A STRING IS A PALINDROME**

```
s = input("Enter a string: ")
if s == s[::-1]:
    print("Palindrome")
else:
    print("Not a palindrome")
```

**OUTPUT:**

Enter a string: RACECAR
Palindrome

**PROGRAM 10: WRITE A PYTHON PROGRAM TO CHECK WHETHER A NUMBER IS AN ARMSTRONG NUMBER**

```
n = int(input("Enter a number: "))
digits = list(map(int, str(n)))
power = len(digits)
if sum(d ** power for d in digits) == n:
    print("Armstrong number")
```

```
else:
    print("Not an Armstrong number")
```

**OUTPUT:**

Enter a number: 153
Armstrong number

**PROGRAM 11: WRITE A PYTHON PROGRAM TO CALCULATE THE FACTORIAL OF A NUMBER USING A LOOP**

```
n = int(input("Enter a number: "))
fact = 1
for i in range(1, n + 1):
    fact *= i
print("Factorial:", fact)
```

**OUTPUT:**

Enter a number: 5
Factorial: 120

**PROGRAM 12: WRITE A PYTHON PROGRAM TO GENERATE THE FIRST N FIBONACCI NUMBERS**

```
n = int(input("Enter n: "))
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()
```

**OUTPUT:**

Enter n: 14
0 1 1 2 3 5 8 13 21 34 55 89 144 233

**PROGRAM 13: WRITE A PYTHON PROGRAM TO DISPLAY THE MULTIPLICATION TABLE OF A NUMBER**

```
n = int(input("Enter a number: "))
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

**OUTPUT:**

Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50

**PROGRAM 14: WRITE A PYTHON PROGRAM TO FIND ALL PRIME NUMBERS UP TO N**

```
n = int(input("Enter n: "))
for num in range(2, n + 1):
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            break
    else:
        print(num, end=" ")
print()
```

**OUTPUT:**

Enter n: 10
2 3 5 7

**PROGRAM 15: WRITE A PYTHON PROGRAM TO COUNT EVEN NUMBERS IN A LIST**

```
lst = list(map(int, input("Enter list elements: ").split()))
count = sum(1 for x in lst if x % 2 == 0)
print("Even count:", count)
```

**OUTPUT:**

Enter list elements: 10 44 22 43 65 72
Even count: 4

**PROGRAM 16: WRITE A PYTHON PROGRAM TO COUNT VOWELS IN A STRING**

```
s = input("Enter a string: ")
vowels = "aeiouAEIOU"
count = sum(1 for ch in s if ch in vowels)
print("Vowel count:", count)
```

**OUTPUT:**

Enter a string: LONDON
Vowel count: 2

**PROGRAM 17: WRITE A PYTHON PROGRAM TO REMOVE DUPLICATE ELEMENTS FROM A LIST**

```
lst = list(map(int, input("Enter list elements: ").split()))
unique = list(set(lst))
print("Without duplicates:", unique)
```

**OUTPUT:**

Enter list elements: 23 43 55 32 55 22 66
Without duplicates: [32, 66, 43, 23, 22, 55]

**PROGRAM 18: WRITE A PYTHON PROGRAM TO CALCULATE WORD FREQUENCY IN A SENTENCE USING A DICTIONARY**

```
sentence = input("Enter a sentence: ")
freq = {}
for word in sentence.split():
    freq[word] = freq.get(word, 0) + 1
print(freq)
```

**OUTPUT:**

```
Enter a sentence: The football team was disciplined by a professional trainer
{'The': 1, 'football': 1, 'team': 1, 'was': 1, 'disciplined': 1, 'by': 1, 'a': 1, 'professional': 1, 'trainer': 1}
```

**PROGRAM 19: WRITE A PYTHON PROGRAM TO DEFINE A FUNCTION THAT ADDS TWO NUMBERS**

```
def add(a, b):
    return a + b

x = int(input("Enter x: "))
y = int(input("Enter y: "))
print("Sum:", add(x, y))
```

**OUTPUT:**

```
Enter x: 10
Enter y: 32
Sum: 42
```

**PROGRAM 20: WRITE A PYTHON PROGRAM TO DEFINE A FUNCTION THAT RETURNS THE MAXIMUM ELEMENT IN A LIST**

```
def max_element(lst):
    return max(lst)

lst = list(map(int, input("Enter list elements: ").split()))
print("Maximum:", max_element(lst))
```

**OUTPUT:**

```
Enter list elements: 33 24 142 55 54 29
Maximum: 142
```

**RESULT:**

Thus, the basic Python programs were successfully executed, demonstrating mathematical operations, string manipulation, list and dictionary processing, conditional statements, loops, and user-defined functions. The expected outputs were obtained successfully.

# EXPERIMENT 2: STRING MANIPULATION AND PATTERN-BASED PROGRAMMING IN PYTHON

**AIM:**

To practice and implement fundamental Python programming exercises involving functions, control structures, list manipulations, string algorithms, and dictionaries.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Write the Python code for each of the given programming exercises.
- Read the required input values from the user using the input() function.
- Use functions and control structures to check prime numbers, generate Fibonacci numbers, calculate GCD, and identify Armstrong numbers.
- Perform list operations such as finding the second-largest element, merging lists without duplicates, rotating lists, and separating even and odd numbers.
- Apply string manipulation techniques to count character types, check anagrams, calculate character frequencies, and find the first non-repeating character.
- Use dictionaries to store character frequencies and student marks, and perform the required operations.
- Implement mathematical algorithms to calculate number conversions, check perfect numbers, and transpose matrices.
- Execute each program by selecting Run → Run Module or pressing F5, and verify the output in the Python Shell.
- Record the program code and corresponding outputs in the lab record.

**PROGRAM 1: WRITE A PYTHON PROGRAM TO CHECK WHETHER A NUMBER IS PRIME USING A FUNCTION**

```
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

n = int(input("Enter a number: "))
print(is_prime(n))
```

**OUTPUT:**

Enter a number: 83
True

**PROGRAM 2: WRITE A PYTHON PROGRAM TO GENERATE ALL FIBONACCI NUMBERS LESS THAN N**

```
n = int(input("Enter n: "))
a, b = 0, 1
while a < n:
    print(a, end=" ")
    a, b = b, a + b
print()
```

**OUTPUT:**

Enter n: 14
0 1 1 2 3 5 8 13

**PROGRAM 3: WRITE A PYTHON PROGRAM TO FIND THE SECOND-LARGEST ELEMENT IN A LIST**

```
lst = list(map(int, input("Enter list elements: ").split()))
largest = max(lst)
second = max(x for x in lst if x != largest)
print(second)
```

**OUTPUT:**

Enter list elements: 28 35 55 53 56 11 45 87
56

**PROGRAM 4: WRITE A PYTHON PROGRAM TO COUNT UPPERCASE, LOWERCASE, DIGIT, AND SPECIAL CHARACTERS IN A STRING**

```
s = input("Enter a string: ")
upper = lower = digit = special = 0

for ch in s:
    if ch.isupper():
        upper += 1
    elif ch.islower():
        lower += 1
    elif ch.isdigit():
        digit += 1
    else:
        special += 1

print(upper, lower, digit, special)
```

**OUTPUT:**

Enter a string: Welcome To the PARK @ Japan10
7 14 2 6

**PROGRAM 5: WRITE A PYTHON PROGRAM TO MERGE TWO LISTS WITHOUT DUPLICATE ELEMENTS**

```
l1 = list(map(int, input("Enter first list: ").split()))
l2 = list(map(int, input("Enter second list: ").split()))

result = []
for x in l1 + l2:
    if x not in result:
        result.append(x)

print(result)
```

**OUTPUT:**

Enter first list: 13 42 55 25 66 76
Enter second list: 43 55 22 55 66 22

[13, 42, 55, 25, 66, 76, 43, 22]

**PROGRAM 6: WRITE A PYTHON PROGRAM TO CHECK WHETHER TWO STRINGS ARE ANAGRAMS**

```
s1 = input("Enter first string: ").replace(" ", "").lower()
s2 = input("Enter second string: ").replace(" ", "").lower()
print(sorted(s1) == sorted(s2))
```

**OUTPUT:**

Enter first string: RACE
Enter second string: CARE
True

**PROGRAM 7: WRITE A PYTHON PROGRAM TO CALCULATE THE GREATEST COMMON DIVISOR (GCD) USING A FUNCTION**

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

print(gcd(12, 18))
```

**OUTPUT:**

6

**PROGRAM 8: WRITE A PYTHON PROGRAM TO RIGHT-ROTATE A LIST BY K POSITIONS**

```
lst = list(map(int, input("Enter list elements: ").split()))
k = int(input("Enter k: ")) % len(lst)
print(lst[-k:] + lst[:-k])
```

**OUTPUT:**

Enter list elements: 24 65 22 53 12 44
Enter k: 22
[22, 53, 12, 44, 24, 65]

**PROGRAM 9: WRITE A PYTHON PROGRAM TO COMPUTE CHARACTER FREQUENCIES USING A DICTIONARY**

```
s = input("Enter a string: ")
freq = {}
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

**OUTPUT:**

Enter a string: Football
{'F': 1, 'o': 2, 't': 1, 'b': 1, 'a': 1, 'l': 2}

**PROGRAM 10: WRITE A PYTHON PROGRAM TO SEPARATE EVEN AND ODD NUMBERS FROM A LIST**

```
lst = list(map(int, input("Enter list elements: ").split()))
even = [x for x in lst if x % 2 == 0]
odd = [x for x in lst if x % 2 != 0]
print(even)
print(odd)
```

**OUTPUT:**

Enter list elements: 12 43 14 55 22 54 31 66 21 56
[12, 14, 22, 54, 66, 56]
[43, 55, 31, 21]

**PROGRAM 11: WRITE A PYTHON PROGRAM TO FIND ALL ARMSTRONG NUMBERS WITHIN A GIVEN RANGE**

```
for num in range(1, 1000):
    order = len(str(num))
```

```
    if num == sum(int(d) ** order for d in str(num)):
        print(num)
```

**OUTPUT:**

```
1
2
3
4
5
6
7
8
9
153
370
371
407
```

**PROGRAM 12: WRITE A PYTHON PROGRAM TO CHECK WHETHER A GIVEN NUMBER IS A PERFECT NUMBER**

```
n = int(input("Enter a number: "))
s = sum(i for i in range(1, n) if n % i == 0)
print(s == n)
```

**OUTPUT:**

```
Enter a number: 432
False
```

**PROGRAM 13: WRITE A PYTHON PROGRAM TO SORT A LIST OF TUPLES BY THEIR SECOND ELEMENT**

```
lst = [(1, 3), (4, 1), (2, 2)]
lst.sort(key=lambda x: x[1])
print(lst)
```

**OUTPUT:**

```
[(4, 1), (2, 2), (1, 3)]
```

**PROGRAM 14: WRITE A PYTHON PROGRAM TO CONVERT A DECIMAL NUMBER TO BINARY, OCTAL, AND HEXADECIMAL FORMATS**

```
n = int(input("Enter a decimal number: "))

def convert(n, base):
    res = ""
    digits = "0123456789ABCDEF"
    while n > 0:
        res = digits[n % base] + res
        n //= base
    return res if res else "0"

print(convert(n, 2), convert(n, 8), convert(n, 16))
```

**OUTPUT:**

Enter a decimal number: 55
110111 67 37

**PROGRAM 15: WRITE A PYTHON PROGRAM TO FIND THE FIRST NON-REPEATING CHARACTER IN A STRING**

```
s = input("Enter a string: ")
for ch in s:
    if s.count(ch) == 1:
        print(ch)
        break
```

**OUTPUT:**

Enter a string: Football
F

**PROGRAM 16: WRITE A PYTHON PROGRAM TO COMPUTE THE TRANSPOSE OF A MATRIX**

```
matrix = [[1, 2, 3], [4, 5, 6]]
transpose = list(map(list, zip(*matrix)))
print(transpose)
```

**OUTPUT:**

[[1, 4], [2, 5], [3, 6]]

**PROGRAM 17: WRITE A PYTHON PROGRAM TO PERFORM PASSWORD VALIDATION USING CRITERIA CHECKS**

```
pwd = input("Enter password: ")
valid = (
    len(pwd) >= 8 and
    any(c.isupper() for c in pwd) and
    any(c.islower() for c in pwd) and
    any(c.isdigit() for c in pwd)
)
print(valid)
```

**OUTPUT:**

Enter password: FCb124322@
True

**PROGRAM 18: WRITE A PYTHON PROGRAM TO FIND THE TOP STUDENT FROM A DICTIONARY OF STUDENT MARKS**

```
students = {"A": 85, "B": 92, "C": 78}
top = max(students, key=students.get)
print(top, students[top])
```

**OUTPUT:**

B 92

**PROGRAM 19: WRITE A PYTHON PROGRAM TO IMPLEMENT A MENU-DRIVEN CALCULATOR USING FUNCTIONS**

```
def add(a, b):
    return a + b
def sub(a, b):
    return a - b
def mul(a, b):
    return a * b
def div(a, b):
```

```
    return a / b if b != 0 else None
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
choice = input("Choose +, -, *, /: ")
if choice == "+":
    print(add(a, b))
elif choice == "-":
    print(sub(a, b))
elif choice == "*":
    print(mul(a, b))
elif choice == "/":
    print(div(a, b))
else:
    print("Invalid choice")
```

**OUTPUT:**

Enter first number: 11.3
Enter second number: 13.4
Choose +, -, *, /: /
0.8432835820895522

**<u>PROGRAM 20:</u> WRITE A PYTHON PROGRAM TO REMOVE DUPLICATE WORDS FROM A SENTENCE**

```
sentence = input("Enter a sentence: ").split()
result = []
for word in sentence:
    if word not in result:
        result.append(word)
print(" ".join(result))
```

**OUTPUT:**

Enter a sentence: Good bye bye world world
Good bye world

**RESULT**:

Thus, Python programming exercises involving functions, lists, strings, dictionaries, matrix operations, and menu-driven applications were successfully executed.

# EXPERIMENT 3:LIST PROCESSING AND USER-DEFINED FUNCTIONS IN PYTHON

**AIM:**
To perform various operations on Python built-in data structures including lists, tuples, and sets for effective data manipulation.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Write the Python code for each of the given list, tuple, and set operations.
- Initialize lists, tuples, and sets with suitable data values.
- Perform list operations such as removing duplicates, sorting elements, reversing lists, copying lists, inserting values, and removing elements.
- Use built-in functions and methods such as min(), max(), sorted(), count(), index(), copy(), insert(), and remove() to manipulate data.
- Perform set operations such as union, intersection, and difference to identify common and distinct elements.
- Execute each program by selecting Run → Run Module or pressing F5.
- Observe and verify the output displayed in the Python Shell.
- Repeat the procedure for all ten programs.
- Record the program code and corresponding outputs in the lab record.

## PROGRAM 1: WRITE A PYTHON PROGRAM TO REMOVE DUPLICATE ELEMENTS FROM A LIST AND SORT IT IN ASCENDING ORDER

```
lst = [7, 3, 8, 7, 2, 3, 1]
result = sorted(list(set(lst)))
print(result)
```

**OUTPUT:**
[1, 2, 3, 7, 8]

**PROGRAM 2: WRITE A PYTHON PROGRAM TO REMOVE THE SMALLEST AND LARGEST ELEMENTS FROM A LIST AND REVERSE THE REMAINING ELEMENTS**

```
lst = [10, 5, 20, 8, 15, 2]
lst.remove(min(lst))
lst.remove(max(lst))
lst.reverse()
print(lst)
```

**OUTPUT:**

[15, 8, 5, 10]

**PROGRAM 3: WRITE A PYTHON PROGRAM TO FIND THE COMMON ELEMENTS OF TWO LISTS**

```
list1 = [1, 2, 3, 4, 5]
list2 = [3, 4, 5, 6, 7]
common = sorted(list(set(list1) & set(list2)))
print(common)
```

**OUTPUT:**

[3, 4, 5]

**PROGRAM 4: WRITE A PYTHON PROGRAM TO COUNT AND REMOVE ALL OCCURRENCES OF A WORD FROM A LIST**

```
words = ["Pele", "Leo", "Andres", "Messi", "Pele"]
word = "Pele"
count = words.count(word)
while word in words:
    words.remove(word)
print("Count:", count)
print("List:", words)
```

**OUTPUT**:

Count: 2
List: ['Leo', 'Andres', 'Messi']

**PROGRAM 5: WRITE A PYTHON PROGRAM TO FIND THE FIRST INDEX AND TOTAL COUNT OF A VALUE IN A TUPLE**

```
tuple1 = (2, 5, 10, 5, 8, 10, 1)
value = 10
print("First Index:", tuple1.index(value))
print("Count:", tuple1.count(value))
```

**OUTPUT**:

First Index: 2
Count: 2

**PROGRAM 6: WRITE A PYTHON PROGRAM TO COPY A LIST AND SORT THE COPY IN DESCENDING ORDER**

```
lst = [8, 2, 5, 9, 10]
copy_lst = lst.copy()
copy_lst.sort(reverse=True)
print("Sorted:", copy_lst)
print("Original:", lst)
```

**OUTPUT**:

Sorted: [10, 9, 8, 5, 2]
Original: [8, 2, 5, 9, 10]

**PROGRAM 7: WRITE A PYTHON PROGRAM TO REMOVE ELEMENTS PRESENT IN A SECOND SET FROM THE FIRST SET**

```
set1 = {1, 2, 3, 4, 5}
set2 = {3, 4, 6, 7}
set1.difference_update(set2)
print(set1)
```

**OUTPUT:**

{1, 2, 5}

**PROGRAM 8: WRITE A PYTHON PROGRAM TO CONVERT A LIST TO A SET, ADD TWO NEW ELEMENTS, AND REMOVE AN EXISTING ELEMENT**

```
lst = [1, 2, 2, 3, 4, 4]
s = set(lst)
s.add(5)
s.add(6)
s.remove(2)
print(s)
```

**OUTPUT**:

{1, 3, 4, 5, 6}

**PROGRAM 9: WRITE A PYTHON PROGRAM TO FIND THE COMMON ELEMENTS ACROSS THREE SETS**

```
set1 = {1, 2, 3, 4}
set2 = {2, 3, 5, 6}
set3 = {1, 2, 3}
common = set1.intersection(set2, set3)
print(common)
```

**OUTPUT:**

{2, 3}

**PROGRAM 10: WRITE A PYTHON PROGRAM TO INSERT A VALUE INTO A LIST AT A SPECIFIED INDEX AND REMOVE ANOTHER VALUE**

```
lst = [10, 20, 30, 40]
lst.insert(2, 25)
lst.remove(40)
print(lst)
```

**OUTPUT:**

[10, 20, 25, 30]

**RESULT:**

Thus, various operations on Python lists, tuples, and sets were successfully executed.

## EXPERIMENT 4: ADVANCED LIST AND STRING PROBLEM SOLVING TECHNIQUES

**AIM:**
To apply advanced problem-solving techniques on Python list and string structures including sublists, rotations, frequency analysis, and matrix operations.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Write the Python code for each of the given list and string problem-solving exercises.
- Initialize lists and strings with suitable input data.
- Apply list operations to find the longest increasing continuous sublist, identify frequently occurring elements, calculate maximum products, and determine the minimum swaps required to sort a list.
- Perform string operations to check rotations, find the longest prefix that is also a suffix, identify words with repeated characters, and find the maximum number after removing one digit.
- Use loops, conditional statements, and built-in functions to implement the required algorithms.
- Execute each program by selecting Run → Run Module or pressing F5.
- Observe and verify the outputs displayed in the Python Shell.
- Repeat the procedure for all ten programs.
- Record the program code and corresponding outputs in the lab record.

### PROGRAM 1: WRITE A PYTHON PROGRAM TO FIND THE LENGTH OF THE LONGEST INCREASING CONTINUOUS SUBLIST

```
lst = [1, 2, 3, 1, 2, 3, 4]
max_len = 1
curr = 1
for i in range(1, len(lst)):
```

```
    if lst[i] > lst[i - 1]:
        curr += 1
        max_len = max(max_len, curr)
    else:
        curr = 1
print(max_len)
```

**OUTPUT:**

4

**<u>PROGRAM 2:</u> WRITE A PYTHON PROGRAM TO CHECK WHETHER ONE STRING IS A ROTATION OF ANOTHER**

```
s1 = "abcd"
s2 = "cdab"

if len(s1) == len(s2) and s2 in (s1 + s1):
    print(True)
else:
    print(False)
```

**OUTPUT:**

True

**<u>PROGRAM 3:</u> WRITE A PYTHON PROGRAM TO FIND ALL ELEMENTS APPEARING MORE THAN N/3 TIMES IN A LIST**

```
lst = [1, 2, 3, 1, 1, 2, 1]
n = len(lst)
result = []

for x in lst:
    if x not in result and lst.count(x) > n // 3:
        result.append(x)

print(result)
```

**OUTPUT:**

[1]

**PROGRAM 4: WRITE A PYTHON PROGRAM TO FIND THE LONGEST PREFIX THAT IS ALSO A SUFFIX IN A STRING**

```
s = "abacaba"
n = len(s)
ans = ""
for i in range(1, n):
    if s[:i] == s[n-i:]:
        ans = s[:i]
print(ans)
```

**OUTPUT:**

Aba

**PROGRAM 5: WRITE A PYTHON PROGRAM TO FIND THE MAXIMUM PRODUCT OF ANY THREE NUMBERS IN A LIST**

```
lst = [-10, -3, 5, 6, -2]
lst.sort()
prod1 = lst[-1] * lst[-2] * lst[-3]
prod2 = lst[0] * lst[1] * lst[-1]
print(max(prod1, prod2))
```

**OUTPUT:**

180

**PROGRAM 6: WRITE A PYTHON PROGRAM TO FIND THE WORD WITH THE MOST REPEATING CHARACTER OCCURRENCES IN A SENTENCE**

```
sentence = "apple banana committee"
words = sentence.split()
max_word = ""
max_count = 0
for w in words:
    count = 0
    for ch in w:
        if w.count(ch) > 1:
            count += 1
    if count > max_count:
```

```
        max_count = count
        max_word = w
print(max_word)
```

**OUTPUT:**

committee

**<u>PROGRAM 7:</u> WRITE A PYTHON PROGRAM TO CALCULATE THE MINIMUM NUMBER OF SWAPS REQUIRED TO SORT A LIST**

```
lst = [4, 3, 2, 1]
swaps = 0
arr = lst[:]
for i in range(len(arr)):
    while arr[i] != i + 1:
        j = arr[i] - 1
        arr[i], arr[j] = arr[j], arr[i]
        swaps += 1
print(swaps)
```

**OUTPUT:**

2

**<u>PROGRAM 8:</u> WRITE A PYTHON PROGRAM TO FIND THE MAXIMUM POSSIBLE NUMBER AFTER REMOVING ONE DIGIT FROM A NUMBER STRING**

```
s = "52743"
best = "0"
for i in range(len(s)):
    temp = s[:i] + s[i+1:]
    if temp > best:
        best = temp
print(best)
```

**OUTPUT:**

5743

**PROGRAM 9: WRITE A PYTHON PROGRAM TO FIND ALL PALINDROMIC SUBSTRINGS IN A GIVEN STRING**

```
s = "ababa"
palindrome = []

for i in range(len(s)):
    for j in range(i + 1, len(s) + 1):
        sub = s[i:j]
        if sub == sub[::-1] and len(sub) > 1:
            palindrome.append(sub)
print(palindrome)
```

**OUTPUT:**

['aba', 'ababa', 'bab', 'aba']

**PROGRAM 10: WRITE A PYTHON PROGRAM TO FIND THE MAXIMUM DIFFERENCE BETWEEN ELEMENTS SUCH THAT THE LARGER ELEMENT APPEARS AT A HIGHER INDEX**

```
lst = [2, 3, 10, 6, 4, 8, 1]

min_val = lst[0]
max_diff = lst[1] - lst[0]

for i in range(1, len(lst)):
    max_diff = max(max_diff, lst[i] - min_val)
    min_val = min(min_val, lst[i])

print(max_diff)
```

**OUTPUT:**

8

**RESULT:**

Thus, advanced list and string problem-solving techniques were successfully implemented and executed.

# EXPERIMENT 5: PYTHON LIST COMPREHENSION

**AIM:**

To implement concise and efficient data processing pipelines in Python using list comprehensions with conditional filtering and transformations.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Write the Python code for each of the given list comprehension exercises.
- Initialize lists, strings, and other required data structures with suitable values.
- Use list comprehensions with conditional filtering to select words containing digits, reverse words, and identify palindromes.
- Apply transformations to square even numbers, cube odd numbers, change character cases, and remove duplicate elements while preserving order.
- Use built-in functions and conditional expressions to filter alphabetic words, sort words by length, and generate abbreviations.
- Execute each program by selecting Run → Run Module or pressing F5.
- Observe and verify the outputs displayed in the Python Shell.
- Repeat the procedure for all ten programs.
- Record the program code and corresponding outputs in the lab record.

## PROGRAM 1: WRITE A PYTHON PROGRAM TO FILTER WORDS THAT CONTAIN AT LEAST ONE DIGIT USING LIST COMPREHENSION

```
words = ["abc", "a1b", "xyz9", "hello "]
result = [w for w in words if any(ch.isdigit() for ch in w)]
print(result)
```

**OUTPUT:**

['a1b', 'xyz9']

**PROGRAM 2: WRITE A PYTHON PROGRAM TO REVERSE WORDS LONGER THAN FOUR CHARACTERS USING LIST COMPREHENSION**

```
sentence = "Python makes programming enjoyable"
words = sentence.split()
result = [w[::-1] for w in words if len(w) > 4]
print(result)
```

**OUTPUT:**

['nohtyP', 'sekam', 'gnimmargorp', 'elbayojne']

**PROGRAM 3: WRITE A PYTHON PROGRAM TO SQUARE EVEN NUMBERS AND CUBE ODD POSITIVE NUMBERS USING LIST COMPREHENSION**

```
nums = [-3, -2, 1, 2, 3, 4]
result = [
    n*n if n % 2 == 0 else n**3
    for n in nums
    if n > 0]
print(result)
```

**OUTPUT:**

[1, 4, 27, 16]

**PROGRAM 4: WRITE A PYTHON PROGRAM TO PERFORM A CASE-INSENSITIVE PALINDROME CHECK ON A LIST OF WORDS USING LIST COMPREHENSION**

```
words = ["Level", "python", "Radar", "hello"]
result = [w for w in words if w.lower() == w.lower()[::-1]]
print(result)
```

**OUTPUT:**

['Level', 'Radar']

**PROGRAM 5: WRITE A PYTHON PROGRAM TO CONVERT VOWELS TO UPPERCASE AND CONSONANTS TO LOWERCASE FOR ALPHABETIC CHARACTERS USING LIST COMPREHENSION**

```
s = "a1B2cDe3"
result = [
    ch.upper() if ch.lower() in "aeiou" else ch.lower()
    for ch in s
    if ch.isalpha()
]
print(result)
```

**OUTPUT:**

['A', 'b', 'c', 'd', 'E']

**PROGRAM 6: WRITE A PYTHON PROGRAM TO FILTER WORDS WITH MATCHING FIRST AND LAST LETTERS AND CONSTRUCT (WORD, LENGTH) TUPLES USING LIST COMPREHENSION**

```
words = ["level", "test", "radar", "python"]
result = [(w, len(w)) for w in words if w[0] == w[-1]]
print(result)
```

**OUTPUT:**

[('level', 5), ('test', 4), ('radar', 5)]

**PROGRAM 7: WRITE A PYTHON PROGRAM TO REMOVE DUPLICATES FROM A LIST WHILE PRESERVING ORDER USING LIST COMPREHENSION**

```
nums = [1, 2, 2, 3, 1, 4, 3]
result = []
[result.append(n) for n in nums if n not in result]
print(result)
```

**OUTPUT:**

[1, 2, 3, 4]

**PROGRAM 8: WRITE A PYTHON PROGRAM TO FILTER PURELY ALPHABETIC WORDS AND CONVERT THEM TO LOWERCASE USING LIST COMPREHENSION**

```
words = ["Hello", "World123", "Python", "AI!"]
result = [w.lower() for w in words if w.isalpha()]
print(result)
```

**OUTPUT:**

['hello', 'python']

**PROGRAM 9: WRITE A PYTHON PROGRAM TO SORT WORDS IN A SENTENCE BY LENGTH IN DESCENDING ORDER**

```
sentence = "Sort these words by length"
result = sorted(sentence.split(), key=len, reverse=True)
print(result)
```

**OUTPUT:**

['length', 'these', 'words', 'Sort', 'by']

**PROGRAM 10: WRITE A PYTHON PROGRAM TO GENERATE FIRST-AND-LAST-LETTER ABBREVIATIONS FOR WORDS OF LENGTH TWO OR MORE USING LIST COMPREHENSION**

```
words = ["apple", "hi", "a", "banana"]
result = [w[0] + w[-1] for w in words if len(w) >= 2]
print(result)
```

**OUTPUT:**

['ae', 'hi', 'ba']

**RESULT**:

Thus, various Python list comprehension techniques were successfully implemented and executed.

# EXPERIMENT 6: DICTIONARY OPERATIONS, FREQUENCY ANALYSIS AND COMPREHENSIONS

**AIM:**
To perform key-value data manipulation, frequency analysis, dictionary filtering, and dictionary comprehensions in Python.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Write the Python code for each of the given dictionary operations and frequency analysis exercises.
- Initialize dictionaries with suitable keys and values for performing data manipulation.
- Calculate character and word frequencies using loops and conditional statements.
- Perform dictionary operations to filter items by value, find the key with the maximum value, square dictionary values, and combine two lists into a dictionary.
- Use dictionary comprehensions to filter values, calculate squares, invert keys and values, and merge dictionaries by retaining larger values.
- Execute each program by selecting Run → Run Module or pressing F5.
- Observe and verify the outputs displayed in the Python Shell.
- Repeat the procedure for all twelve programs.
- Record the program code and corresponding outputs in the lab record.

## PROGRAM 1: WRITE A PYTHON PROGRAM TO COUNT CHARACTER FREQUENCIES IN A STRING USING A DICTIONARY

```
text = "dictionary"
char_count = {}

for char in text:
    if char in char_count:
        char_count[char] += 1
    else:
        char_count[char] = 1
```

```
print(char_count)
```

**OUTPUT:**

{'d': 1, 'i': 2, 'c': 1, 't': 1, 'o': 1, 'n': 1, 'a': 1, 'r': 1, 'y': 1}

**PROGRAM 2: WRITE A PYTHON PROGRAM TO FILTER DICTIONARY ITEMS BY VALUE**

```
scores = {"A": 4, "B": 68, "C": 98, "D": 40}
filtered_scores = {}
for key, value in scores.items():
    if value > 50:
        filtered_scores[key] = value
print(filtered_scores)
```

**OUTPUT:**

{'B': 68, 'C': 98}

**PROGRAM 3: WRITE A PYTHON PROGRAM TO FIND THE KEY ASSOCIATED WITH THE MAXIMUM VALUE IN A DICTIONARY**

```
marks = {"Siri": 98, "Vendhan": 100,"Vikki": 88, "Vaisu": 99}
max_key = No
max_value = 0
for key, value in marks.items():
    if value > max_value:
        max_value = value
        max_key = key
print(max_key)
```

**OUTPUT:**

Vendhan

**PROGRAM 4: WRITE A PYTHON PROGRAM TO SQUARE EACH VALUE IN A DICTIONARY**

```
nums = {"a": 2, "b": 4, "c": 6}
squared_nums = {}
```

```
for key, value in nums.items():
    squared_nums[key] = value ** 2

print(squared_nums)
```

**OUTPUT**:

{'a': 4, 'b': 16, 'c': 36}

**PROGRAM 5: WRITE A PYTHON PROGRAM TO COMBINE TWO LISTS INTO A DICTIONARY AS KEYS AND VALUES**

```
keys = ["id", "name", "age"]
values = [10, "Leo", 20]
combined_dict = {}
for i in range(len(keys)):
    combined_dict[keys[i]] = values[i]
print(combined_dict)
```

**OUTPUT:**

{'id': 10, 'name': 'Leo', 'age': 20}

**PROGRAM 6: WRITE A PYTHON PROGRAM TO COUNT WORD FREQUENCY IN A SENTENCE USING A DICTIONARY**

```
sentence = "Playing football is simple, but playing simple
football is the hardest"
words = sentence.split()
word_count = {}
for word in words:
    if word in word_count:
        word_count[word] += 1
    else:
        word_count[word] = 1
print(word_count)
```

**OUTPUT:**

{'Playing': 1, 'football': 2, 'is': 2, 'simple,': 1, 'but': 1, 'playing': 1, 'simple': 1, 'the': 1, 'hardest': 1}

**PROGRAM 7: WRITE A PYTHON PROGRAM TO SWAP KEYS AND VALUES IN A DICTIONARY**

```
data = {"a": 1, "b": 2, "c": 3}
swapped_data = {}
for key, value in data.items():
    swapped_data[value] = key

print(swapped_data)
```

**OUTPUT:**

{1: 'a', 2: 'b', 3: 'c'}

**PROGRAM 8: WRITE A PYTHON PROGRAM TO CALCULATE THE SUM OF ALL VALUES IN A DICTIONARY**

```
expenses = {"Food": 1000,"Rent": 6000,"Travel": 2400}
total = sum(expenses.values())
print(total)
```

**OUTPUT:**

9400

**PROGRAM 9: WRITE A PYTHON PROGRAM TO FILTER DICTIONARY ITEMS BY VALUE USING DICTIONARY COMPREHENSION**

```
scores = {"A": 46, "B": 78, "C": 88, "D": 40}
filtered_scores = {key: value for key, value in scores.items() if
value > 50}
print(filtered_scores)
```

**OUTPUT:**

{'B': 78, 'C': 88}

**PROGRAM 10: WRITE A PYTHON PROGRAM TO SQUARE ALL VALUES IN A DICTIONARY USING DICTIONARY COMPREHENSION**

```
numbers = {"a": 2, "b": 3, "c": 4}
squared_numbers = {key: value ** 2 for key, value in
numbers.items()}
print(squared_numbers)
```

**OUTPUT:**

{'a': 4, 'b': 9, 'c': 16}

**PROGRAM 11: WRITE A PYTHON PROGRAM TO INVERT KEYS AND VALUES IN A DICTIONARY USING DICTIONARY COMPREHENSION**

```
data = {"a": 1, "b": 2, "c": 3}
inv_data = {value: key for key, value in data.items()}
print(inv_data)
```

**OUTPUT:**

{1: 'a', 2: 'b', 3: 'c'}

**PROGRAM 12: WRITE A PYTHON PROGRAM TO MERGE TWO DICTIONARIES RETAINING THE LARGER VALUE FOR DUPLICATE KEYS USING DICTIONARY COMPREHENSION**

```
d1 = {"a": 10, "b": 20, "c": 30}
d2 = {"b": 25, "c": 15, "d": 40}
all_keys = set(d1) | set(d2)
merged = { key: max(d1.get(key, 0), d2.get(key, 0)) for key in
all_keys}
 print(merged)
```

**OUTPUT:**

{'b': 25, 'd': 40, 'a': 10, 'c': 30}

**RESULT**:

Thus, dictionary operations, frequency analysis, and dictionary comprehensions were successfully executed.

# EXPERIMENT 7: GENERATOR EXPRESSIONS AND LAZY EVALUATION TECHNIQUES IN PYTHON

**AIM:**
To utilize generator expressions and lazy evaluation techniques in Python for memory-efficient computation and data processing.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Write the Python code for each of the given generator expression exercises.
- Use generator expressions with functions such as sum() to perform memory-efficient calculations.
- Calculate the sum of fourth powers, count numbers with repeated digits, and identify words satisfying specified length conditions.
- Generate Pythagorean triples and calculate sums based on divisibility conditions.
- Apply generator expressions to perform matrix calculations, count words containing exactly two vowels, flatten dictionary values, calculate factorial sums, and count numbers whose squares end in six.
- Execute each program by selecting Run → Run Module or pressing F5.
- Observe and verify the outputs displayed in the Python Shell.
- Repeat the procedure for all ten programs.
- Record the program code and corresponding outputs in the lab record.

## PROGRAM 1: WRITE A PYTHON PROGRAM TO CALCULATE THE SUM OF FOURTH POWERS OF NUMBERS WHOSE DIGIT SUM IS DIVISIBLE BY 5 USING A GENERATOR EXPRESSION

```
result = sum(n**4 for n in range(1, 101) if sum(int(d) for d in
str(n)) % 5 == 0)
print(result)
```

**OUTPUT:**

387559626

**PROGRAM 2: WRITE A PYTHON PROGRAM TO COUNT NUMBERS WITH REPEATED DIGITS USING A GENERATOR EXPRESSION**

```
count = sum(1 for n in range(1, 501) if len(set(str(n))) <
len(str(n)))
print(count)
```

**OUTPUT:**

122

**PROGRAM 3: WRITE A PYTHON PROGRAM TO COUNT WORDS HAVING A LENGTH GREATER THAN 6 USING A GENERATOR EXPRESSION**

```
sen = ["He kicked the football across the grass",
    "She watches a football match every weekend",
    "The Olympics is the ultimate athletic tournament"]
count = sum(1 for s in sen for word in s.split() if len(word) > 6)
print(count)
```

**OUTPUT:**

8

**PROGRAM 4: WRITE A PYTHON PROGRAM TO GENERATE PYTHAGOREAN TRIPLES USING A GENERATOR EXPRESSION**

```
triples = ((x, y, z)
    for x in range(1, 6)
    for y in range(1, 6)
    for z in range(1, 6)
    if x*x + y*y == z*z)
print(list(triples))
```

**OUTPUT:**

[(3, 4, 5), (4, 3, 5)]

**PROGRAM 5: WRITE A PYTHON PROGRAM TO CALCULATE THE SUM OF NUMBERS DIVISIBLE BY 3 OR 7 BUT NOT BOTH USING A GENERATOR EXPRESSION**

```
result = sum(n for n in range(200) if (n % 3 == 0) ^ (n % 7 == 0))
print(result)
```

**OUTPUT:**

7585

**PROGRAM 6: WRITE A PYTHON PROGRAM TO COMPUTE THE SUM OF ELEMENTS ABOVE THE MAIN DIAGONAL OF A MATRIX USING A GENERATOR EXPRESSION**

```
matrix = [[2,4,6],[1,3,5],[7,8,9]]

total = sum(
    matrix[i][j]
    for i in range(len(matrix))
    for j in range(len(matrix))
    if j > i)
print(total)
```

**OUTPUT:**

15

**PROGRAM 7: WRITE A PYTHON PROGRAM TO COUNT WORDS CONTAINING EXACTLY TWO VOWELS USING A GENERATOR EXPRESSION**

```
words = ["quantum", "gravity", "friction", "atom", "velocity",
"energy"]
vowels = "aeiou"

count = sum(1 for w in words if sum(1 for ch in w if ch in vowels)
== 2)
 print(count)
```

**OUTPUT:**

3

**PROGRAM 8: WRITE A PYTHON PROGRAM TO FLATTEN NESTED DICTIONARY LIST VALUES AND COMPUTE THEIR TOTAL SUM USING A GENERATOR EXPRESSION**

```
data = {
    "A": [1,2,3],
    "B": [4,5],
    "C": [6,7,8]}
total = sum(x for values in data.values() for x in values)
print(total)
```

**OUTPUT:**

36

**PROGRAM 9: WRITE A PYTHON PROGRAM TO CALCULATE THE SUM OF FACTORIALS FROM 1 TO 8 USING A GENERATOR EXPRESSION**

```
import math
result = sum(math.factorial(n) for n in range(1, 9))
print(result)
```

**OUTPUT:**

46233

**PROGRAM 10: WRITE A PYTHON PROGRAM TO COUNT NUMBERS WHOSE SQUARE ENDS WITH 6 USING A GENERATOR EXPRESSION**

```
count = sum(1 for n in range(1, 301) if (n*n) % 10 == 6)
Print(count)
```

**OUTPUT:**

60

**RESULT**:

Thus, generator expressions and lazy evaluation techniques in Python were successfully implemented and executed.

# EXPERIMENT 8: MATRIX OPERATIONS USING NUMPY

**AIM:**

To construct, reshape, slice, and perform mathematical and matrix operations using the NumPy library in Python.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Install the NumPy library if it is not already installed.
- Import NumPy using the statement import numpy as np.
- Create matrices using NumPy functions such as np.arange(), np.array(), np.zeros(), np.eye(), and np.random.randint().
- Perform mathematical operations on matrix elements, including calculating squares, cubes, row sums, and column sums.
- Implement matrix multiplication, block matrix construction, matrix reshaping, and matrix slicing using appropriate NumPy functions.
- Calculate square roots and perform other numerical operations on matrix elements.
- Execute each program by selecting Run → Run Module or pressing F5.
- Observe and verify the outputs displayed in the Python Shell.
- Repeat the procedure for all ten programs and record the program code and corresponding outputs in the lab record.

## PROGRAM 1: WRITE A PYTHON PROGRAM TO CALCULATE ELEMENT-WISE SQUARES AND CUBES OF A MATRIX USING NUMPY

```
import numpy as np
A = np.arange(1, 10).reshape(3, 3)
square = A ** 2
cube = A ** 3
print(square)
print(cube)
```

**OUTPUT:**

```
 [[ 1  4  9]
  [16 25 36]
  [49 64 81]]
[[  1   8  27]
 [ 64 125 216]
 [343 512 729]]
```

**<u>PROGRAM 2:</u> WRITE A PYTHON PROGRAM TO CALCULATE ROW SUMS AND COLUMN SUMS OF A MATRIX USING NUMPY**

```
import numpy as np

A = np.arange(1, 17).reshape(4, 4)
row_sum = np.sum(A, axis=1)
col_sum = np.sum(A, axis=0)
print(row_sum)
print(col_sum)
```

**OUTPUT:**

```
[10 26 42 58]
[28 32 36 40]
```

**<u>PROGRAM 3:</u> WRITE A PYTHON PROGRAM TO CONSTRUCT A BLOCK MATRIX FROM 2X2 MATRICES USING NUMPY**

```
import numpy as np
A = np.array([[1, 2],[3, 4]])
B = np.array([[5, 6],[7, 8]])
C = np.block([[A, B],[B, A]])
print(C)
```

**OUTPUT:**

```
[[1 2 5 6]
 [3 4 7 8]
 [5 6 1 2]
 [7 8 3 4]]
```

**PROGRAM 4: WRITE A PYTHON PROGRAM TO PERFORM MATRIX MULTIPLICATION USING NUMPY**

```
import numpy as np

A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[7, 8], [9, 10], [11, 12]])
C = np.matmul(A, B)
print(C)
```

**OUTPUT:**

```
[[ 58  64]
 [139 154]]
```

**PROGRAM 5: WRITE A PYTHON PROGRAM TO GENERATE A RANDOM 5x5 MATRIX, COMPUTE ROW/COLUMN SUMS, AND CALCULATE ELEMENT SQUARES USING NUMPY**

```
import numpy as np

A = np.random.randint(1, 51, size=(5, 5))
row_sum = np.sum(A, axis=1)
col_sum = np.sum(A, axis=0)
square = A ** 2

print(A)
print("Row sums :",row_sum)
print("Column sums: ", col_sum)
print("Square of all elements :\n",square)
```

**OUTPUT:**

```
[[ 9 26 12 26 50]
 [46  9 10 19 41]
 [ 5 48 39 30 47]
 [25  7 16  1  7]
 [40 36 10 30 16]]
```

Row sums : [123 125 169  56 132]

Column sums:  [125 126  87 106 161]

Square of all elements :

```
[[  81  676  144  676 2500]
 [2116   81  100  361 1681]
```

[ 25 2304 1521 900 2209]
[ 625 49 256 1 49]
[1600 1296 100 900 256]]

**PROGRAM 6: WRITE A PYTHON PROGRAM TO RESHAPE A MATRIX AND COMPUTE ELEMENT-WISE CUBES USING NUMPY**

```
import numpy as np
A = np.arange(1, 13).reshape(3, 4)
cube = A ** 3
print(A)
print(cube)
```

**OUTPUT:**

[[ 1 2 3 4]
[ 5 6 7 8]
[ 9 10 11 12]]

[[ 1 8 27 64]
[ 125 216 343 512]
[ 729 1000 1331 1728]]

**PROGRAM 7: WRITE A PYTHON PROGRAM TO PERFORM MATRIX MULTIPLICATION ON TWO RANDOM MATRICES USING NUMPY**

```
import numpy as np
A = np.random.randint(1, 10, size=(3, 3))
B = np.random.randint(1, 10, size=(3, 3))
C = np.matmul(A, B)
print(A)
print(B)
print(C)
```

**OUTPUT:**

[[4 4 9]
[2 6 4]
[2 1 3]]

[[1 5 1]
[4 1 3]
[8 2 2]]

[[92 42 34]
[58 24 28]
[30 17 11]]

**PROGRAM 8: WRITE A PYTHON PROGRAM TO PERFORM MATRIX SLICING USING NUMPY**

```
import numpy as np
A = np.arange(1, 17).reshape(4, 4)
extracted = A[0:2, 2:4]
print(extracted)
```

**OUTPUT:**

[[3 4]
[7 8]]

**PROGRAM 9: WRITE A PYTHON PROGRAM TO CONSTRUCT A BLOCK MATRIX USING IDENTITY AND ZERO MATRICES IN NUMPY**

```
import numpy as np

I = np.eye(2)
Z = np.zeros((2, 2))
A = np.block([[Z, I],[I, Z]])
print(I)
print(Z)
print(A)
```

**OUTPUT:**

[[1. 0.]
[0. 1.]]

[[0. 0.]
[0. 0.]]

[[0. 0. 1. 0.]
[0. 0. 0. 1.]
[1. 0. 0. 0.]
[0. 1. 0. 0.]]

**<u>PROGRAM 10:</u> WRITE A PYTHON PROGRAM TO GENERATE A RANDOM MATRIX, COMPUTE STATISTICS, CALCULATE SQUARE ROOTS, AND PERFORM RESHAPING USING NUMPY**

```
import numpy as np
A = np.random.rand(4, 4)
row_sum = np.sum(A, axis=1)
col_sum = np.sum(A, axis=0)
sqrt_A = np.sqrt(A)
B = A.reshape(2, 8)

print(A)
print("Row sums :",row_sum)
print("Column sums :",col_sum)
print("Square root of elements :", sqrt_A)
print("16 Reshaped (2 x 8 ) :\n", B)
```

**OUTPUT:**

```
[[0.63890974 0.77294699 0.34591252 0.59220326]
 [0.00885123 0.04760661 0.7395532  0.49262547]
 [0.9384987  0.33713621 0.92254327 0.87369315]
 [0.93385074 0.07724485 0.86337842 0.59822017]]

Row sums : [2.34997252 1.2886365  3.07187134 2.47269419]

Column sums : [2.52011041 1.23493466 2.87138742 2.55674206]

Square root of elements : [[0.7993183  0.87917404 0.58814328 0.76954744]
 [0.09408096 0.21818938 0.85997279 0.70187283]
 [0.96876143 0.58063431 0.96049116 0.93471555]
 [0.96635953 0.27792958 0.92918159 0.77344694]]

16 Reshaped (2 x 8 ) : [[0.63890974 0.77294699 0.34591252 0.59220326 0.00885123 0.04760661
  0.7395532  0.49262547]
 [0.9384987  0.33713621 0.92254327 0.87369315 0.93385074 0.07724485
  0.86337842 0.59822017]]
```

**RESULT**:

Thus, numerical computing and matrix operations using NumPy were successfully implemented and executed.

# EXPERIMENT 9: ADVANCED NUMERICAL SIMULATION AND STATISTICAL ANALYSIS USING NUMPY AND GENERATOR EXPRESSIONS

**AIM:**
To perform advanced numerical simulations, Monte-Carlo estimations, and statistical analyses using NumPy vectorized functions and generator expressions.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Import the NumPy library using the statement import numpy as np.
- Generate random sensor readings and normalize the data using the mean and standard deviation.
- Perform numerical calculations using generator expressions to compute the sum of squares of non-zero matrix elements.
- Estimate the value of Pi using the Monte Carlo simulation method.
- Generate random matrices and analyze their stability, sliding windows, covariance, mean, variance, and median.
- Calculate correlations between feature vectors and perform Principal Component Analysis (PCA) using eigenvalues and eigenvectors.
- Apply NumPy broadcasting to calculate spare-part billing and identify the group with maximum revenue.
- Execute each program by selecting Run → Run Module or pressing F5.
- Observe the outputs and record the program code and corresponding results in the lab record.

**<u>PROGRAM 1:</u> WRITE A PYTHON PROGRAM TO GENERATE RANDOM SENSOR READINGS AND PERFORM Z-SCORE NORMALIZATION USING NUMPY**

```
import numpy as np

# Generate 1000 random numbers
data = np.fromiter((np.random.rand() for _ in range(1000)),
dtype=float)

# Compute mean and standard deviation of original data
mean = np.mean(data)
std = np.std(data)

# Normalize the data
normalized = (data - mean) / std

# Print results
print("Original Mean =", mean)
print("Original Standard Deviation =", std)
print("Normalized Data =", normalized)

# Check mean and standard deviation of normalized data
print("Normalized Mean =", np.mean(normalized))
print("Normalized Standard Deviation =", np.std(normalized))
print(np.isclose(np.mean(normalized), 0))
print(round(np.mean(normalized), 10))
```

**OUTPUT:**

Original Mean = 0.49971810922709675
Original Standard Deviation = 0.2842653206510492
Normalized Data = [ 0.9167914  0.07112228  0.07230593  0.79548011  1.31980338  0.44610435
 -1.25301288 -0.9627841  -0.06610858 -0.28911292 -1.65461842  1.72123451
 ... ... ... ... ... ... ]
Normalized Mean = -1.4210854715202004e-16
Normalized Standard Deviation = 0.9999999999999999
True
-0.0

**PROGRAM 2: WRITE A PYTHON PROGRAM TO CALCULATE THE SUM OF SQUARES OF NON-ZERO MATRIX ELEMENTS USING NUMPY AND GENERATOR EXPRESSIONS**

```
import numpy as np
A = np.random.randint(-5, 5, (50, 50))
energy = sum(x**2 for x in A.flatten() if x != 0)
print("Energy:", energy)
```

**OUTPUT:**

Energy: 21826

**PROGRAM 3: WRITE A PYTHON PROGRAM TO ESTIMATE THE VALUE OF PI USING A MONTE-CARLO SIMULATION WITH GENERATOR EXPRESSIONS**

```
import numpy as np
N = 10000
points = ((np.random.rand(), np.random.rand()) for _ in range(N))
inside = sum(1 for x, y in points if x*x + y*y <= 1)
pi_est = 4 * inside / N
print("Estimated Pi:", pi_est)
```

**OUTPUT:**

Estimated Pi: 3.1224

**PROGRAM 4: WRITE A PYTHON PROGRAM TO CHECK AND SELECT STABLE RANDOM 3X3 MATRICES USING NUMPY EIGENVALUES**

```
import numpy as np
# Generate 10 random 3x3 matrices
matrices = [np.random.randn(3, 3) for _ in range(10)]

# Select only stable matrices
stable = (A for A in matrices if
np.all(np.abs(np.linalg.eigvals(A)) < 1))

# Print each stable matrix
for s in stable:
    print("Stable Matrix:\n", s)
```

**OUTPUT:**

```
Stable Matrix:
 [[-0.12160575  1.22208327  1.1181796 ]
 [ 0.08467328  0.26837808  0.62413821]
 [ 0.03297314 -0.82322296 -0.45067356]]
Stable Matrix:
 [[ 0.42161264 -2.33694794 -0.51724673]
 [ 0.69495697 -1.38556699  0.01764607]
 [ 0.10648996  0.3335019  -0.07646256]]
Stable Matrix:
 [[-1.26347729 -0.1886716  -1.20636105]
 [ 0.6159641   0.35491193 -0.3426939 ]
 [ 1.35576208  0.0924897   1.11108649]]
Stable Matrix:
 [[-0.05547822  0.03493985 -0.08953179]
 [ 0.85332948 -0.29617823  0.8290937 ]
 [ 0.08480479  0.05359313  0.11481479]]
```

**<u>PROGRAM 5:</u> WRITE A PYTHON PROGRAM TO COMPUTE COVARIANCES ACROSS SLIDING WINDOWS USING NUMPY**

```
import numpy as np

data = np.random.rand(200)
windows = (data[i:i+10] for i in range(len(data)-9))
covariances = [np.cov(w) for w in windows]
print(covariances)
```

**OUTPUT:**

```
[array(0.04398058), array(0.04704257), array(0.0587587), array(0.06969343), array(0.07295164),
 array(0.09980489), array(0.09865325), array(0.10413644), array(0.10403226), array(0.10110106),
 array(0.080983), array(0.07567821), array(0.06618755), array(0.07338818), array(0.07988177),
 array(0.06490199), array(0.08415385), array(0.07825554), array(0.08 ... [truncated]
```

**<u>PROGRAM 6:</u> WRITE A PYTHON PROGRAM TO COMPUTE THE MEAN, VARIANCE, AND MEDIAN OF 500 RANDOM VALUES USING NUMPY**

```
import numpy as np
# Generate 500 random numbers using a generator expression
data = np.fromiter((np.random.rand() for _ in range(500)),
```

```
dtype=float)

# Compute mean, variance, and median
print("Mean:", np.mean(data))
print("Variance:", np.var(data))
print("Median:", np.median(data))
```

**OUTPUT:**

Mean: 0.5095692889905568
Variance: 0.08186255562905541
Median: 0.5241443916371349

**PROGRAM 7: WRITE A PYTHON PROGRAM TO FIND PAIRS OF HIGHLY CORRELATED FEATURE VECTORS USING NUMPY**

```
import numpy as np
from itertools import combinations
vectors = np.random.rand(20, 5)
pairs = ((i, j) for i, j in combinations(range(20), 2)
         if np.corrcoef(vectors[i], vectors[j])[0, 1] > 0.9)
print(list(pairs))
```

**OUTPUT:**

[(1, 12), (2, 5), (7, 8), (9, 15)]

**PROGRAM 8: WRITE A PYTHON PROGRAM TO PERFORM PRINCIPAL COMPONENT ANALYSIS (PCA) USING NUMPY**

```
import numpy as np
X = np.random.rand(100, 5)
X_centered = X - np.mean(X, axis=0)
cov = np.cov(X_centered, rowvar=False)
eigvals, eigvecs = np.linalg.eig(cov)
pc1 = eigvecs[:, np.argmax(eigvals)]
projection = X_centered @ pc1
print(projection)
```

**OUTPUT:**

```
[ 0.05354788 -0.6042408   0.15566616  0.14370678 -0.30169168  0.09145753
 -0.51408497 -0.23984711  0.46116215 -0.37141207  0.02963886 -0.23193134
 ……….    ……….    ……….    ……….    ……….
  0.48844465  0.24380641 -0.27464471 -0.73445571]
```

**PROGRAM 9: WRITE A PYTHON PROGRAM TO FIND THE MOST INFLUENTIAL NODE FROM THE DOMINANT EIGENVECTOR OF A MATRIX USING NUMPY**

```
import numpy as np
A = np.random.randint(0, 2, (6, 6))
eigvals, eigvecs = np.linalg.eig(A)
dominant = eigvecs[:, np.argmax(eigvals)]
node = max((i for i in range(len(dominant))),
           key=lambda i: abs(dominant[i]))
print("Most influential node:", node)
```

**OUTPUT:**

Most influential node: 5

**PROGRAM 10: WRITE A PYTHON PROGRAM TO CALCULATE ROBOT SPARE-PART BILLING AND MAXIMUM GROUP REVENUE USING NUMPY BROADCASTING**

```
import numpy as np
S = np.array([[320, 360, 400],
              [280, 310, 350],
              [450, 480, 520],
              [240, 270, 300]])
d = np.array([20, 25, 30])
t = np.array([[0.05], [0.08], [0.10], [0.06]])
final = (S - d) * (1 + t)
revenue = final.sum(axis=1)
best = np.argmax(revenue)
print("Shape:", final.shape)
print(final)
print("Revenue:", revenue)
print("Maximum group:", best + 1)
```

**OUTPUT:**

```
Shape: (4, 3)
[[315.   351.75 388.5 ]
 [280.8  307.8  345.6 ]
 [473.   500.5  539.  ]
 [233.2  259.7  286.2 ]]
Revenue: [1055.25  934.2  1512.5   779.1 ]
Maximum group: 3
```

**RESULT**:

Thus, advanced numerical simulations and statistical analyses using NumPy and generator expressions were successfully executed.

## EXPERIMENT 10: LINEAR ALGEBRA COMPUTATIONS USING NUMPY LINALG PACKAGE

**AIM:**
To perform linear algebra computations, matrix inversions, eigenvalue decompositions, and system solving using the NumPy linalg package.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Import the NumPy library using the statement import numpy as np.
- Perform numerical calculations using generator expressions and NumPy vectorization, and compare their results.
- Create matrices and calculate determinants, matrix inverses, and verify the results using identity matrices.
- Identify singular matrices by checking whether their determinants are zero.
- Compute eigenvalues and eigenvectors, solve systems of linear equations, and determine matrix rank and linear dependence.
- Perform matrix multiplication, construct block diagonal matrices, and verify determinant properties and orthogonality of eigenvectors.
- Compare the execution time of Python loops and NumPy vectorized operations.
- Execute each program by selecting Run → Run Module or pressing F5, and verify the outputs in the Python Shell.
- Record the program code and corresponding outputs in the lab record.

### PROGRAM 1: WRITE A PYTHON PROGRAM TO COMPARE NUMERICAL COMPUTATION RESULTS BETWEEN A GENERATOR EXPRESSION AND NUMPY VECTORIZATION

```
import numpy as np
# Generator expression
gen_sum = sum(i*i/(i+1) for i in range(1, 51))
```

```
# NumPy vectorized solution
i = np.arange(1, 51)
numpy_sum = np.sum(i*i/(i+1))

print("Generator sum =", gen_sum)
print("NumPy sum     =", numpy_sum)
print("Equal?        =", np.isclose(gen_sum, numpy_sum))
```

**OUTPUT:**

Generator sum = 1228.5188131814666
NumPy sum = 1228.5188131814666
Equal? = True

**<u>PROGRAM 2:</u> WRITE A PYTHON PROGRAM TO CHECK INVERTIBILITY, COMPUTE DETERMINANT, AND FIND THE INVERSE OF A MATRIX USING NUMPY**

```
import numpy as np

A = np.random.rand(4, 4)
det_A = np.linalg.det(A)
print("A =\n", A)
print("det(A) =", det_A)
if abs(det_A) > 1e-10:
    A_inv = np.linalg.inv(A)
    product = np.dot(A, A_inv)
    print("A inverse =\n", A_inv)
    print("A @ A_inverse =\n", product)
    print("Verified?", np.allclose(product, np.eye(4)))
else:
    print("Matrix is singular; inverse does not exist.")
```

**OUTPUT:**

A =
[[0.93352767 0.08711062 0.01650631 0.41502475]
[0.91450689 0.72761105 0.731582 0.11746699]
[0.57095853 0.75642696 0.9606909 0.87270042]
[0.52358598 0.32920579 0.72120133 0.60873597]]
det(A) = -0.1737531524374657
A inverse =
[[ 0.76081788 0.50883091 -0.82054516 0.55945605]
[ 0.16416108 0.44706268 2.55092378 -3.85526462]

[-1.22873844  0.4881618  -1.73281131  3.22773616]
[ 0.71257731 -1.25777894  1.37917407 -0.57758292]]
A @ A_inverse =
[[ 1.00000000e+00  4.73214434e-17  3.64956297e-17 -2.40106545e-17]
[-7.46875954e-17  1.00000000e+00  7.71734079e-17  1.35071266e-16]
[-2.15448356e-16  4.19361394e-18  1.00000000e+00 -8.09369782e-17]
[-2.21265981e-17  8.16503293e-17 -4.63741803e-18  1.00000000e+00]]
Verified? True

**<u>PROGRAM 3:</u> WRITE A PYTHON PROGRAM TO IDENTIFY A SINGULAR MATRIX WITH A ZERO DETERMINANT USING NUMPY**

```
import numpy as np

A = np.array([[1, 2, 3],
              [2, 4, 6],
              [1, 1, 1]], dtype=float)
det_A = np.linalg.det(A)
print("A =\n", A)
print("det(A) =", det_A)
print("Numerically zero?", np.isclose(det_A, 0))
if np.isclose(det_A, 0):
    print("A is singular, so A inverse does not exist.")
```

**OUTPUT:**

A =
[[1. 2. 3.]
[2. 4. 6.]
[1. 1. 1.]]
det(A) = 0.0
Numerically zero? True
A is singular, so A inverse does not exist.

**<u>PROGRAM 4:</u> WRITE A PYTHON PROGRAM TO COMPUTE AND VERIFY EIGENVVALUES AND EIGENVECTORS USING NUMPY**

```
import numpy as np
A = np.array([[4, 2],[1, 3]], dtype=float)
values, vectors = np.linalg.eig(A)
print("Eigenvalues =", values)
```

```
print("Eigenvectors =\n", vectors)

for k in range(len(values)):
    v = vectors[:, k]
    lam = values[k]
    print("\nEigenvalue:", lam)
    print("A v      =", np.dot(A, v))
    print("lambda v =", lam * v)
    print("Verified?", np.allclose(np.dot(A, v), lam * v))
```

**OUTPUT:**

Eigenvalues = [5. 2.]
Eigenvectors =
[[ 0.89442719 -0.70710678]
[ 0.4472136  0.70710678]]

Eigenvalue: 5.0
A v     = [4.47213595 2.23606798]
lambda v = [4.47213595 2.23606798]
Verified? True

Eigenvalue: 2.0
A v     = [-1.41421356  1.41421356]
lambda v = [-1.41421356  1.41421356]
Verified? True

**<u>PROGRAM 5:</u> WRITE A PYTHON PROGRAM TO SOLVE A LINEAR SYSTEM WITH NON-UNIQUE SOLUTIONS USING LEAST SQUARES IN NUMPY**

```
import numpy as np
A = np.array([[1, 1, 1],[2, 2, 2]], dtype=float)
b = np.array([3, 6], dtype=float)

x, residuals, rank, singular_values = np.linalg.lstsq(A, b,
rcond=None)

print("One NumPy solution =", x)
print("A @ x =", A @ x)
print("b     =", b)
print("Rank(A) =", rank)
print("Residuals =", residuals)
```

```
if rank < A.shape[1]:
    print("No unique solution: infinitely many solutions exist.")
```

**OUTPUT:**

One NumPy solution = [1. 1. 1.]
A @ x = [3. 6.]
b = [3. 6.]
Rank(A) = 1
Residuals = []
No unique solution: infinitely many solutions exist.

**PROGRAM 6: WRITE A PYTHON PROGRAM TO COMPUTE MATRIX RANK AND DETERMINE LINEAR DEPENDENCE OF ROWS USING NUMPY**

```
import numpy as np

A = np.array([[1, 2], [2, 4]])
rank_A = np.linalg.matrix_rank(A)
print("A =\n", A)
print("Rank(A) =", rank_A)

if rank_A < min(A.shape):
    print("Rows/columns are linearly dependent.")
    print("Here Row 2 = 2 * Row 1.")
```

**OUTPUT:**

A =
[[1 2]
[2 4]]
Rank(A) = 1
Rows/columns are linearly dependent.
Here Row 2 = 2 * Row 1.

**PROGRAM 7: WRITE A PYTHON PROGRAM TO DEMONSTRATE THE NON-COMMUTATIVITY OF MATRIX MULTIPLICATION USING NUMPY**

```
import numpy as np
A = np.random.rand(3, 3)
B = np.random.rand(3, 3)
AB = np.dot(A, B)
BA = np.dot(B, A)
```

```
print("A =\n", A)
print("B =\n", B)
print("AB =\n", AB)
print("BA =\n", BA)
print("AB = BA ?", np.allclose(AB, BA))
```

**OUTPUT:**

```
A =
 [[0.88439146 0.71999591 0.54379594]
 [0.4179787  0.58227894 0.88014685]
 [0.52197976 0.9751439  0.22849014]]
B =
 [[0.95477082 0.17344784 0.5736331 ]
 [0.01153925 0.02993232 0.93773703]
 [0.06496347 0.97805581 0.09198301]]
AB =
 [[0.88802625 0.70680972 1.23250303]
 [0.46297033 0.9507592  0.8667495 ]
 [0.52446699 0.34320069 1.23487063]]
BA =
 [[1.21631353 1.34780093 0.80292958]
 [0.51219604 0.94016572 0.2468835 ]
 [0.51427291 0.7059714  0.91717682]]
AB = BA ? False
```

**<u>PROGRAM 8:</u> WRITE A PYTHON PROGRAM TO CONSTRUCT A BLOCK DIAGONAL MATRIX AND VERIFY ITS DETERMINANT PROPERTY USING NUMPY**

```
import numpy as np

A = np.array([[1, 2],[3, 4]])
B = np.array([[5, 6],[7, 8]])
Z = np.zeros((2, 2))
M = np.block([[A, Z],[Z, B]])
det_M = np.linalg.det(M)
product = np.linalg.det(A) * np.linalg.det(B)

print("Block matrix M =\n", M)
print("det(M) =", det_M)
print("det(A) det(B) =", product)
print("Verified?", np.isclose(det_M, product))
```

**OUTPUT:**

```
Block matrix M =
 [[1. 2. 0. 0.]
 [3. 4. 0. 0.]
 [0. 0. 5. 6.]
 [0. 0. 7. 8.]]
det(M) = 4.0000000000000115
det(A) det(B) = 4.000000000000011
Verified? True
```

**PROGRAM 9: WRITE A PYTHON PROGRAM TO VERIFY THE ORTHOGONALITY OF EIGENVECTORS FOR A SYMMETRIC MATRIX USING NUMPY**

```
import numpy as np

A = np.array([[2, 1],[1, 2]], dtype=float)
values, vectors = np.linalg.eig(A)
v1 = vectors[:, 0]
v2 = vectors[:, 1]
dot_product = np.dot(v1, v2)

print("Eigenvalues =", values)
print("Eigenvectors =\n", vectors)
print("v1 . v2 =", dot_product)
print("Orthogonal?", np.isclose(dot_product, 0))
```

**OUTPUT:**

```
Eigenvalues = [3. 1.]
Eigenvectors =
 [[ 0.70710678 -0.70710678]
 [ 0.70710678  0.70710678]]
v1 . v2 = 0.0
Orthogonal? True
```

**PROGRAM 10: WRITE A PYTHON PROGRAM TO COMPARE PERFORMANCE BETWEEN PYTHON LOOPS AND NUMPY VECTORIZATION**

```
import numpy as np
import time
A = np.random.rand(1000, 1000)
```

```
# NumPy vectorization
start = time.time()
B = A * A
numpy_time = time.time() - start

# Python nested loops / list comprehension
start = time.time()
C = [[A[i, j] * A[i, j] for j in range(1000)]
     for i in range(1000)]
loop_time = time.time() - start

print("NumPy time =", numpy_time, "seconds")
print("Loop time =", loop_time, "seconds")
print("Same result?", np.allclose(B, np.array(C)))
if numpy_time > 0:
    print("Speed-up factor =", loop_time / numpy_time)
```

**OUTPUT:**

NumPy time = 0.004002809524536133 seconds
Loop time = 0.47124505043029785 seconds
Same result? True
Speed-up factor = 117.7285722794687

**RESULT:**

Thus, linear algebra computations using the NumPy linalg package were successfully implemented and executed.

## EXPERIMENT 11: DATA CLEANING, PREPROCESSING AND VISUALIZATION PIPELINE USING CSV DATASETS

**AIM:**
To implement a complete data cleaning, preprocessing, and visualization pipeline on CSV datasets using Python for effective exploratory data analysis.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Import the required libraries: Pandas, NumPy, and Matplotlib.
- Prepare the CSV datasets for students, sales, employees, health, and orders, and load them using pd.read_csv().
- Clean column names and standardize text values by removing unnecessary spaces and converting text to lowercase.
- Handle missing values using median values for numerical columns and mode values for categorical columns.
- Remove duplicate records and encode categorical values into numerical representations.
- Detect and handle outliers using the Interquartile Range (IQR) method.
- Perform feature engineering by creating additional columns such as total marks, profit ratio, salary per experience, BMI, and order efficiency where applicable.
- Save the cleaned datasets as CSV files and display their shapes and column names.
- Execute the program and verify the results in the Python Shell.

**PROGRAM: WRITE A PYTHON PROGRAM TO IMPLEMENT A COMPLETE DATA CLEANING, PREPROCESSING, AND FEATURE ENGINEERING PIPELINE ON CSV DATASETS**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

files = {
    "students": "students.csv",
```

```
    "sales": "sales.csv",
    "employees": "employees.csv",
    "health": "health.csv",
    "orders": "orders.csv"}

def clean_columns(df):
    df.columns = df.columns.str.strip().str.lower().str.replace("
", "_")
    return df

def standardize_text(df):
    for col in df.select_dtypes(include='object').columns:
        df[col] = df[col].astype(str).str.strip().str.lower()
    return df

def handle_missing(df):
    for col in df.columns:
        if df[col].dtype in ['float64', 'int64']:
            df[col] = df[col].fillna(df[col].median())
        else:
            df[col] = df[col].replace("none", np.nan)
            df[col] = df[col].fillna(df[col].mode()[0])
    return df

def remove_duplicates(df):
    return df.drop_duplicates()

def encode_categorical(df):
    for col in df.select_dtypes(include='object').columns:
        df[col] = df[col].astype('category').cat.codes
    return df

def handle_outliers(df):
    numeric_cols = df.select_dtypes(include=['int64',
'float64']).columns
    for col in numeric_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df[col] = np.where(df[col] < lower, lower,
                           np.where(df[col] > upper, upper,
df[col]))
```

```
    return df

def feature_engineering(df, name):
    if name == "students":
        if all(col in df.columns for col in ["maths", "physics",
"chemistry"]):
            df["total_marks"] = df["maths"] + df["physics"] +
df["chemistry"]

    elif name == "sales":
        if "sales" in df.columns and "profit" in df.columns:
            df["profit_ratio"] = df["profit"] / (df["sales"] + 1)

    elif name == "employees":
        if "salary" in df.columns and "experience" in df.columns:
            df["salary_per_exp"] = df["salary"] /
(df["experience"] + 1)

    elif name == "health":
        if "weight" in df.columns and "height" in df.columns:
            df["bmi"] = df["weight"] / ((df["height"] / 100) ** 2)

    elif name == "orders":
        if "amount" in df.columns and "deliverytime" in
df.columns:
            df["efficiency"] = df["amount"] / (df["deliverytime"]
+ 1)

    return df

# MAIN PIPELINE
for name, path in files.items():
    print(f"\n========== Processing {name.upper()} ==========")

    df = pd.read_csv(path)

    # Part A & B: Cleaning and Standardization
    df = clean_columns(df)
    df = standardize_text(df)
    df = handle_missing(df)
    df = remove_duplicates(df)
    df = encode_categorical(df)

    # Part C: Outlier Handling
```

```
    df = handle_outliers(df)

    # Part D: Feature Engineering
    df = feature_engineering(df, name)

    # Save cleaned file
    df.to_csv(f"cleaned_{name}.csv", index=False)

    print("Shape after cleaning:", df.shape)
    print("Columns:", list(df.columns))
print("\nPIPELINE COMPLETED SUCCESSFULLY")
```

**OUTPUT:**

======= Processing STUDENTS =======

| student_n | gender | maths | physics | chemistry | city | total_mark |
|---|---|---|---|---|---|---|
| 1 | 3 | 78 | 82 | 75 | 0 | 235 |
| 12 | 1 | 65 | 59 | 70 | 0 | 194 |
| 6 | 2 | 74.5 | 74 | 69 | 2 | 217.5 |
| 3 | 1 | 88 | 91 | 85 | 2 | 264 |
| 13 | 3 | 92 | 70 | 88 | 1 | 250 |
| 15 | 0 | 55 | 60 | 70 | 1 | 185 |
| 18 | 3 | 74.5 | 45 | 50 | 3 | 169.5 |
| 0 | 1 | 70 | 68 | 66 | 3 | 204 |
| 16 | 2 | 60 | 55 | 58 | 4 | 173 |
| 9 | 1 | 95 | 89 | 92 | 4 | 276 |
| 5 | 3 | 40 | 38 | 42.75 | 2 | 120.75 |
| 7 | 1 | 85 | 80 | 78 | 1 | 243 |
| 10 | 2 | 72 | 75 | 70 | 3 | 217 |
| 11 | 1 | 68 | 70 | 65 | 4 | 203 |
| 4 | 3 | 74.5 | 64 | 60 | 0 | 198.5 |
| 2 | 1 | 90 | 70 | 87 | 2 | 247 |
| 14 | 2 | 58 | 62 | 70 | 1 | 190 |
| 17 | 1 | 77 | 73 | 74 | 3 | 224 |
| 8 | 3 | 63 | 59 | 61 | 4 | 183 |

======= Processing SALES =======

| region | sales | profit | product | profit_ratio |
|---|---|---|---|---|
| 1 | 12000 | 1800 | 0 | 0.149988 |
| 2 | 8500 | 1200 | 1 | 0.14116 |
| 0 | 15000 | 2200 | 0 | 0.146657 |
| 3 | 9800 | 1400 | 2 | 0.142843 |
| 1 | 11200 | 1600 | 1 | 0.142844 |
| 2 | 7600 | 1100 | 0 | 0.144718 |
| 0 | 11200 | 900 | 2 | 0.08035 |
| 3 | 13400 | 2000 | 1 | 0.149243 |
| 1 | 9900 | 1650 | 0 | 0.16665 |
| 2 | 10500 | 1500 | 2 | 0.142844 |
| 0 | 12500 | 1900 | 1 | 0.151988 |
| 3 | 8900 | 1300 | 0 | 0.146051 |
| 1 | 11200 | 1700 | 2 | 0.151772 |
| 2 | 14200 | 2100 | 1 | 0.147877 |
| 0 | 9600 | 1400 | 0 | 0.145818 |
| 3 | 11800 | 1750 | 2 | 0.148293 |
| 1 | 10200 | 1550 | 1 | 0.151946 |
| 2 | 8700 | 1250 | 0 | 0.143662 |
| 0 | 15600 | 2300 | 2 | 0.147426 |
| 3 | 17187.5 | 2750 | 0 | 0.159991 |

===== Processing EMPLOYEES =====

| employee | departme | salary | experienc | salary_per_exp |
|---|---|---|---|---|
| 0 | 2 | 45000 | 2 | 15000 |
| 1 | 1 | 38000 | 5 | 6333.333 |
| 2 | 3 | 52000 | 8 | 5777.778 |
| 3 | 2 | 43000 | 3 | 10750 |
| 4 | 1 | 41000 | 3.5 | 9111.111 |
| 5 | 3 | 36000 | 1 | 18000 |
| 6 | 2 | 56000 | 10.125 | 5033.708 |
| 7 | 0 | 47000 | 4 | 9400 |
| 8 | 2 | 39000 | 2 | 13000 |
| 9 | 1 | 43000 | 6 | 6142.857 |
| 10 | 2 | 48000 | 3 | 12000 |
| 11 | 3 | 37000 | 1 | 18500 |
| 12 | 1 | 44000 | 5 | 7333.333 |
| 13 | 2 | 46000 | 4 | 9200 |
| 14 | 0 | 43000 | 7 | 5375 |
| 15 | 3 | 35000 | 1 | 17500 |
| 16 | 1 | 42000 | 3 | 10500 |
| 17 | 2 | 49000 | 6 | 7000 |
| 18 | 0 | 40000 | 2 | 13333.33 |

====== Processing HEALTH =======

| patient_id | gender | weight | height | bmi |
|---|---|---|---|---|
| 0 | 3 | 70 | 172 | 23.66144 |
| 1 | 1 | 58 | 160 | 22.65625 |
| 2 | 2 | 82 | 178 | 25.88057 |
| 3 | 1 | 65 | 165 | 23.87511 |
| 4 | 3 | 90 | 180 | 27.77778 |
| 5 | 0 | 55 | 158 | 22.03173 |
| 6 | 3 | 65.5 | 168 | 23.2072 |
| 7 | 1 | 68 | 162 | 25.91068 |
| 8 | 2 | 72 | 175 | 23.5102 |
| 9 | 1 | 60 | 159 | 23.73324 |
| 10 | 3 | 75 | 164.5 | 27.71593 |
| 11 | 3 | 50 | 155 | 20.81165 |
| 12 | 1 | 88 | 182 | 26.56684 |
| 13 | 2 | 63 | 163 | 23.71184 |
| 14 | 1 | 59 | 161 | 22.76147 |
| 15 | 3 | 92.25 | 170 | 31.92042 |
| 16 | 1 | 66 | 164 | 24.53896 |
| 17 | 2 | 54 | 157 | 21.90758 |
| 18 | 1 | 71 | 173 | 23.72281 |
| 19 | 3 | 62 | 166 | 22.49964 |

======= Processing ORDERS =======

| order_id | payment_ | amount | deliverytir | efficiency |
|---|---|---|---|---|
| 0 | 2 | 500 | 2 | 166.6667 |
| 1 | 2 | 1200 | 4 | 240 |
| 2 | 0 | 850 | 3 | 212.5 |
| 3 | 0 | 870 | 1 | 435 |
| 4 | 1 | 1890 | 5 | 315 |
| 5 | 1 | 600 | 2 | 200 |
| 6 | 0 | 750 | 3 | 187.5 |
| 7 | 0 | 980 | 2.5 | 280 |
| 8 | 1 | 1450 | 4 | 290 |
| 9 | 2 | 300 | 1 | 150 |
| 10 | 0 | 620 | 2 | 206.6667 |
| 11 | 2 | 890 | 3 | 222.5 |
| 12 | 1 | 1100 | 4 | 220 |
| 13 | 0 | 870 | 2 | 290 |
| 14 | 2 | 540 | 1 | 270 |
| 15 | 1 | 760 | 3 | 190 |
| 16 | 2 | 910 | 2 | 303.3333 |
| 17 | 0 | 1330 | 4 | 266 |
| 18 | 1 | 480 | 1 | 240 |
| 19 | 2 | 1890 | 3 | 472.5 |

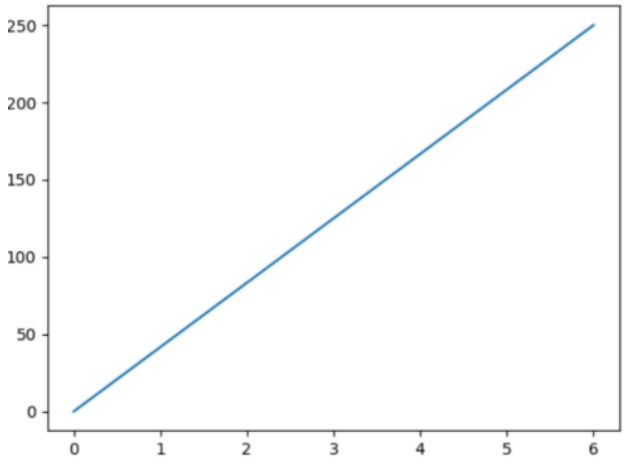
PIPELINE COMPLETED SUCCESSFULLY

**RESULT**:

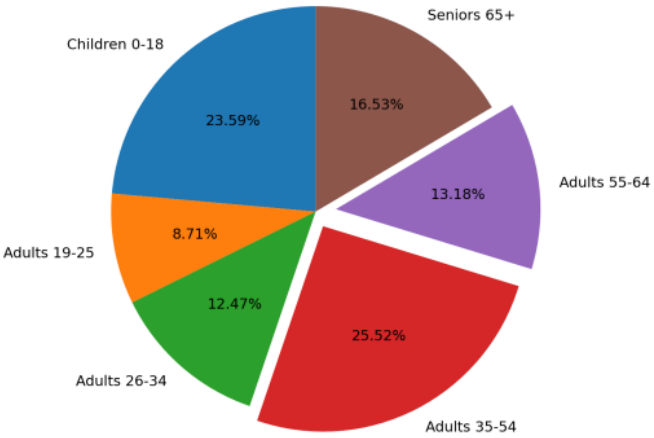
Thus, the complete data cleaning, preprocessing, and visualization pipeline using CSV datasets was successfully implemented and executed.

# EXPERIMENT 12: DATA VISUALIZATION USING LINE, BAR, SCATTER AND PIE CHARTS

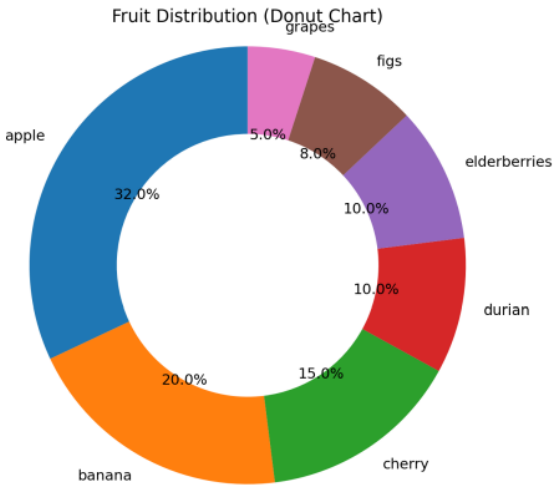
**1. Write the code to obtain the following diagrams:**



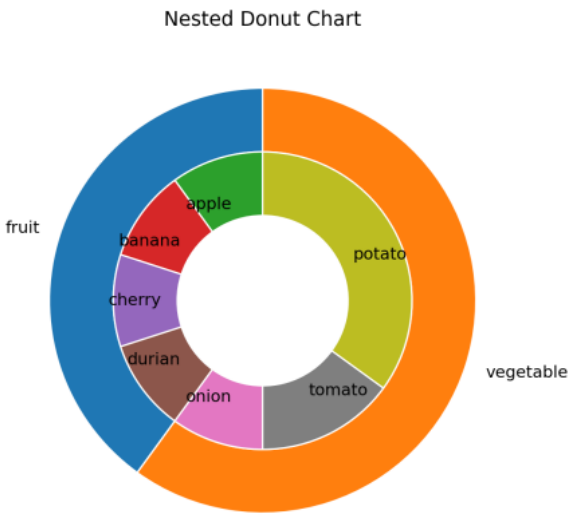

a) Line Plot How to Find the Slope *m*




(b) Pie Chart




(c) Donut Chart




(d) Nested Donut Chart

**AIM:**

To visualize data graphically using line plots, bar charts, scatter plots, and pie charts in Python for effective analysis and interpretation of datasets.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Import NumPy and Matplotlib using the statements import numpy as np and import matplotlib.pyplot as plt.
- Prepare suitable datasets containing numerical values, categories, and labels for visualization.
- Generate line plots, bar charts, scatter plots, and pie charts using the appropriate Matplotlib functions.
- Create donut charts and nested donut charts to represent categorical data and proportions.
- Generate random integer datasets and visualize them using line plots, scatter plots, vertical bar charts, and horizontal bar charts.
- Generate random samples from binomial, exponential, normal, and uniform distributions and display their histograms.
- Add suitable titles, axis labels, legends, and other necessary chart properties.
- Execute the program by selecting Run → Run Module or pressing F5, and observe the generated plots.
- Record the program code and corresponding graphical outputs in the lab record.

**<u>PROGRAM 1:</u> WRITE A PYTHON PROGRAM TO GENERATE LINE, PIE, DONUT, AND NESTED DONUT CHARTS USING MATPLOTLIB**

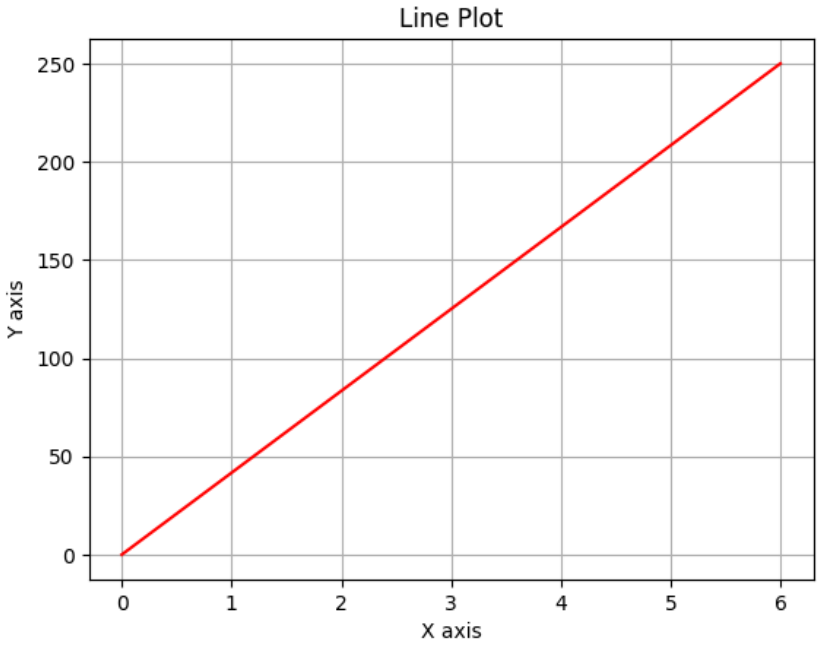
```
import numpy as np
import matplotlib.pyplot as plt

# Line Plot
x = np.arange(0, 7)
m = 250 / 6
y = m * x
```

```
plt.plot(x, y, color='red')
plt.grid(True)
plt.title('Line Plot')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.show()
```

**OUTPUT:**

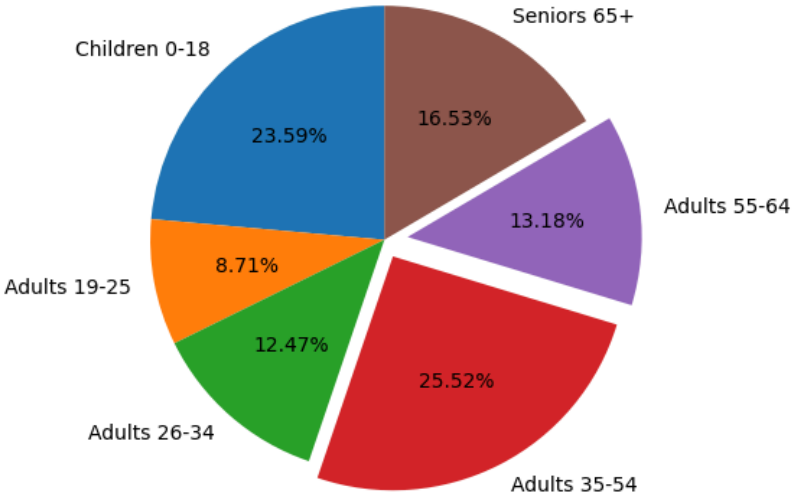
<Figure size 640x480 with 1 Axes>




```
# Pie Chart
labels = ['Children 0-18', 'Adults 19-25', 'Adults 26-34', 'Adults
35-54', 'Adults 55-64', 'Seniors 65+']
sizes = [23.59, 8.71, 12.47, 25.52, 13.18, 16.53]
explode = [0, 0, 0, 0.08, 0.10, 0]
plt.figure()
plt.pie(sizes, labels=labels, autopct='%1.2f%%', explode=explode,
startangle=90)
plt.axis('equal')
plt.show()
```

**OUTPUT:**

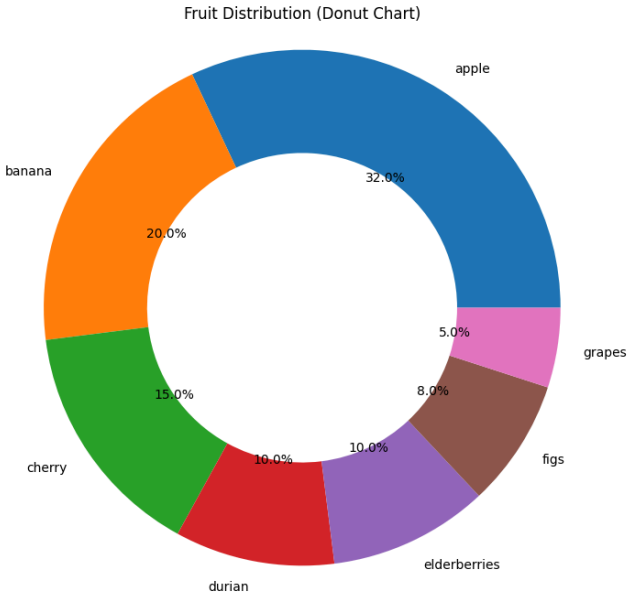
<Figure size 640x480 with 1 Axes>




```
# Donut Chart
labels = ['apple', 'banana', 'cherry', 'durian', 'elderberries',
'figs', 'grapes']
sizes = [32, 20, 15, 10, 10, 8, 5]
plt.figure(figsize=(8, 8))
plt.pie(sizes, labels=labels, autopct='%1.1f%%',
wedgeprops=dict(width=0.4))
plt.title('Fruit Distribution (Donut Chart)')
plt.axis('equal')
plt.show()
```

**OUTPUT:**

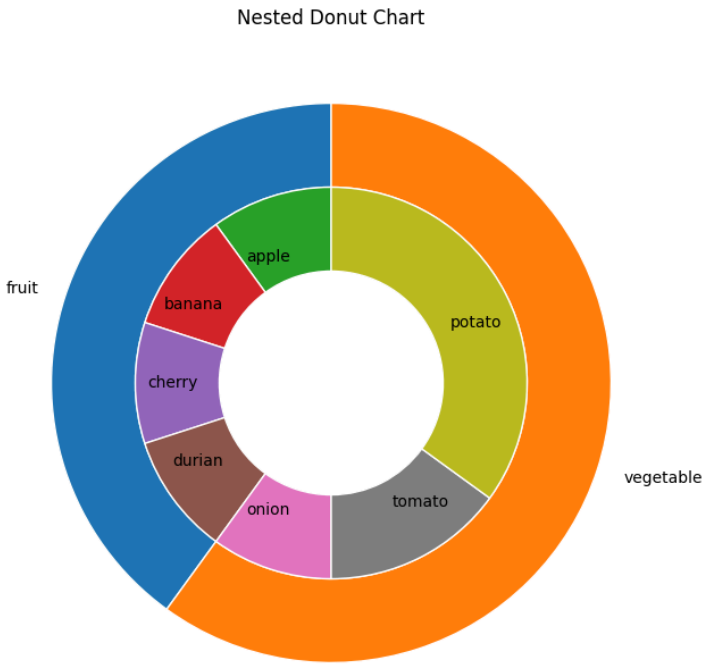
<Figure size 800x800 with 1 Axes>



```
#Nested Donut Chart
outer_labels = ['fruit', 'vegetable']
outer_sizes = [40, 60]
inner_labels = ['apple', 'banana', 'cherry', 'durian', 'onion',
'tomato', 'potato']
inner_sizes = [10, 10, 10, 10, 10, 15, 35]
fig, ax = plt.subplots(figsize=(8, 8))
ax.pie(outer_sizes, labels=outer_labels, radius=1, startangle=90,
wedgeprops=dict(width=0.3, edgecolor='white'))
ax.pie(inner_sizes, labels=inner_labels, radius=0.7, startangle=90,
labeldistance=0.68, wedgeprops=dict(width=0.3, edgecolor='white'))
ax.set(aspect='equal')
plt.title('Nested Donut Chart')
plt.show()
```

**OUTPUT:**

<Figure size 800x800 with 1 Axes>




**<u>PROGRAM 2:</u> WRITE A PYTHON PROGRAM TO GENERATE 100 RANDOM INTEGERS AND PLOT LINE, SCATTER, BAR, AND HORIZONTAL BAR CHARTS USING MATPLOTLIB**

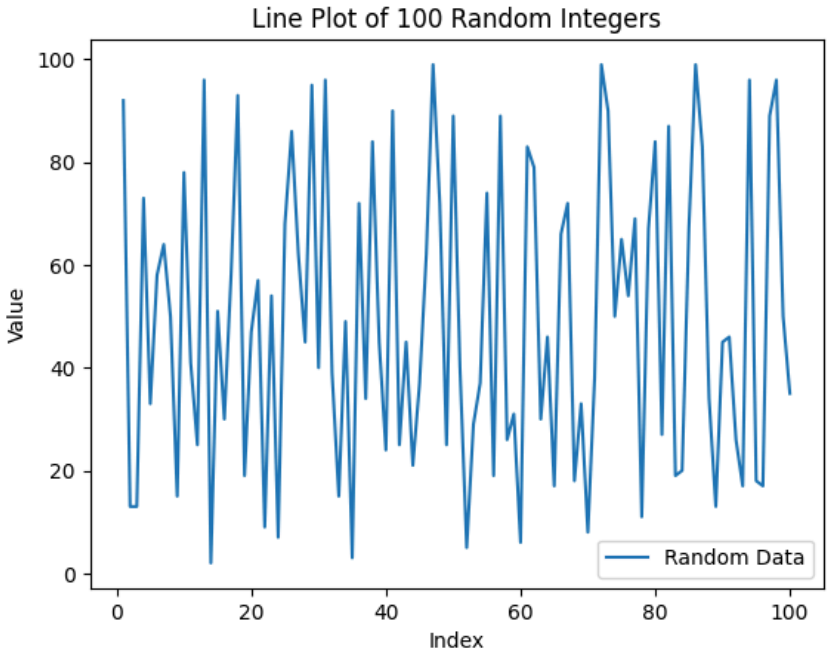
**(a) Line plot**

```
import numpy as np
import matplotlib.pyplot as plt
```

```
data = np.random.randint(1, 100, 100)
x = np.arange(1, 101)
plt.figure()
plt.plot(x, data, label='Random Data')
plt.title('Line Plot of 100 Random Integers')
plt.xlabel('Index')
plt.ylabel('Value')
plt.legend()
plt.show()
```

**OUTPUT:**

<Figure size 640x480 with 1 Axes>

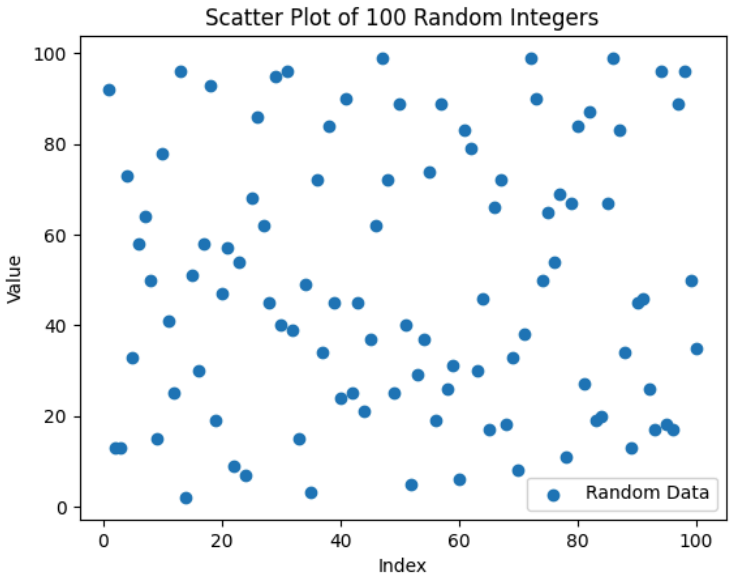



**(b) Scatter plot**

```
plt.figure()
plt.scatter(x, data, label='Random Data')
plt.title('Scatter Plot of 100 Random Integers')
plt.xlabel('Index')
plt.ylabel('Value')
plt.legend()
plt.show()
```

**OUTPUT:**

<Figure size 640x480 with 1 Axes>

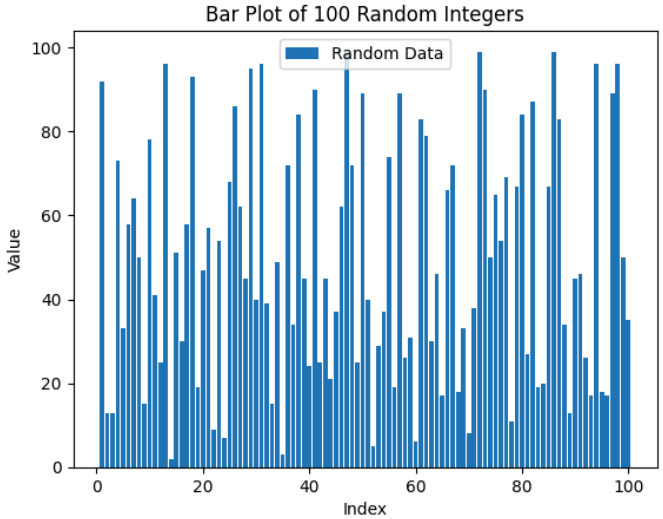



**(c) Bar plot**

```
plt.figure()
plt.bar(x, data, label='Random Data')
plt.title('Bar Plot of 100 Random Integers')
plt.xlabel('Index')
plt.ylabel('Value')
plt.legend()
plt.show()
```

**OUTPUT:**

<Figure size 640x480 with 1 Axes>

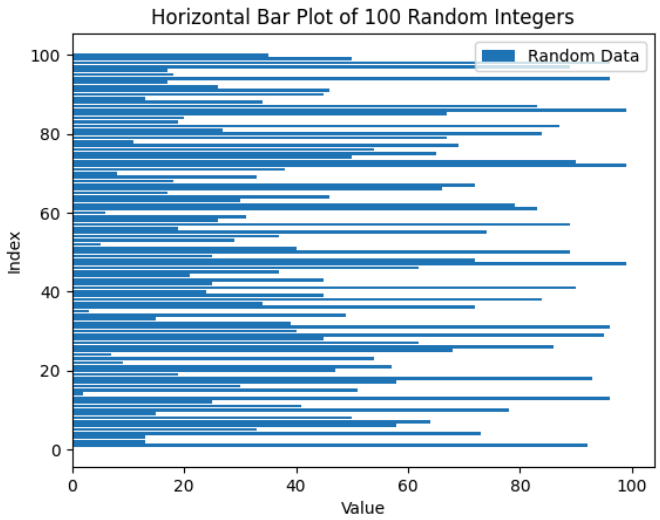


**(d) Horizontal bar plot**

```
plt.figure()
plt.barh(x, data, label='Random Data')
plt.title('Horizontal Bar Plot of 100 Random Integers')
plt.xlabel('Value')
plt.ylabel('Index')
plt.legend()
plt.show()
```

**OUTPUT:**

<Figure size 640x480 with 1 Axes>




**<u>PROGRAM 3:</u> WRITE A PYTHON PROGRAM TO PLOT HISTOGRAMS FOR BINOMIAL, EXPONENTIAL, NORMAL, AND UNIFORM DISTRIBUTIONS USING MATPLOTLIB**

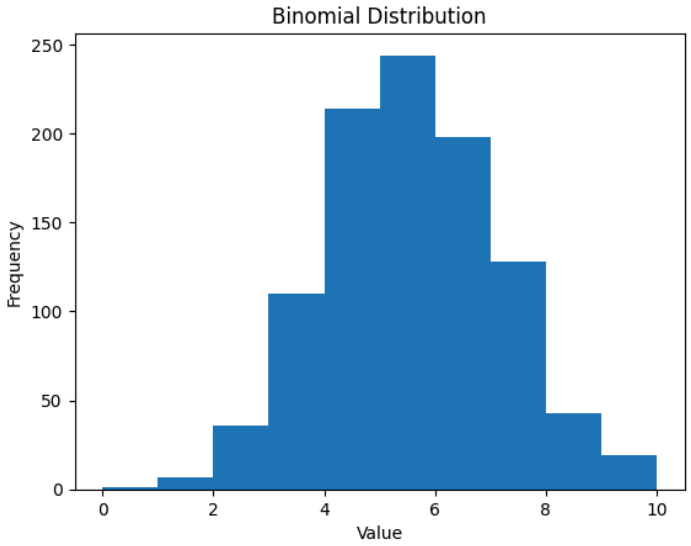
**(a) Binomial distribution**

```
import numpy as np
import matplotlib.pyplot as plt

binomial = np.random.binomial(n=10, p=0.5, size=1000)
plt.figure()
plt.hist(binomial)
plt.title('Binomial Distribution')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```

**OUTPUT:**

<Figure size 640x480 with 1 Axes>

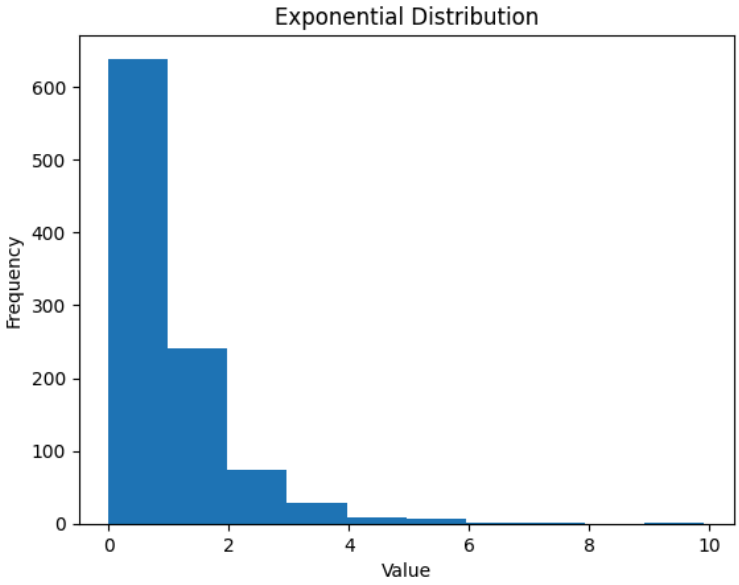



**(b). Exponential distribution**

```
expo = np.random.exponential(scale=1.0, size=1000)
plt.figure()
plt.hist(expo)
plt.title('Exponential Distribution')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```

**OUTPUT:**

<Figure size 640x480 with 1 Axes>

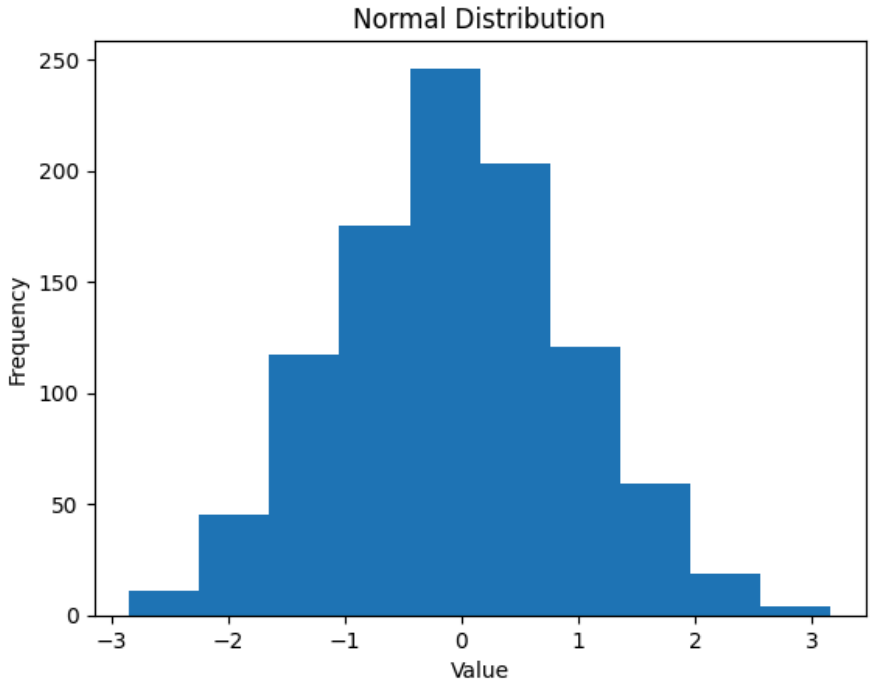


**(c). Normal distribution**

```
normal = np.random.normal(loc=0, scale=1, size=1000)
plt.figure()
plt.hist(normal)
plt.title('Normal Distribution')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```

**OUTPUT:**

<Figure size 640x480 with 1 Axes>

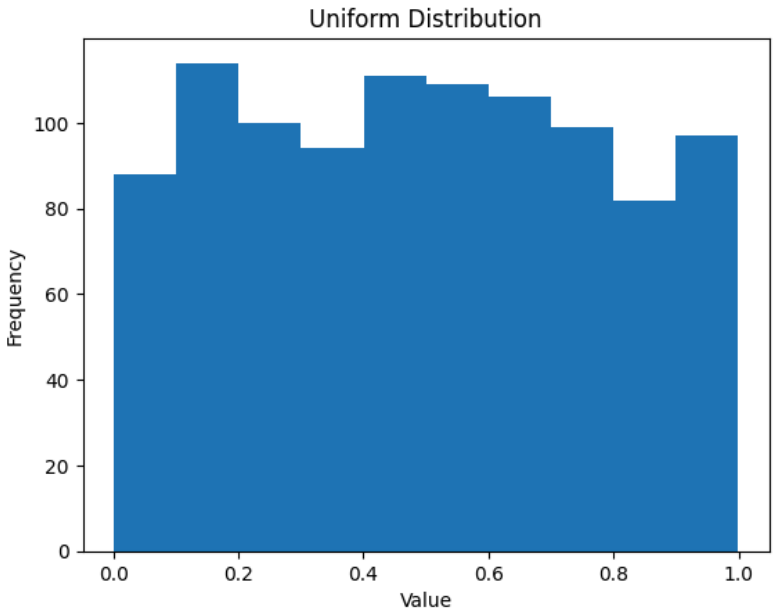



**(d) Uniform distribution**

```
uniform = np.random.uniform(low=0, high=1, size=1000)
plt.figure()
plt.hist(uniform)
plt.title('Uniform Distribution')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```

**OUTPUT:**

<Figure size 640x480 with 1 Axes>




**<u>PROGRAM 4:</u> WRITE A PYTHON PROGRAM TO PLOT GIVEN DATASETS USING LINE PLOTS, BAR CHARTS, SCATTER PLOTS, PIE CHARTS, AND MULTIPLE LINE PLOTS USING MATPLOTLIB**

**a) x1 = [1, 2, 3, 4, 5, 6, 7]**
**y1 = [10, 15, 13, 18, 20, 22, 25]**

**b) x2 = ['A', 'B', 'C', 'D', 'E']**
**y2 = [25, 40, 30, 35, 20]**

**c) x3 = [5, 10, 15, 20, 25, 30, 35]**
**y3 = [12, 18, 20, 28, 30, 33, 40]**

**d) labels4 = ['Rent', 'Food', 'Transport', 'Savings']**
**sizes4 = [40, 25, 15, 20]**

**e) x5 = [1, 2, 3, 4, 5]**
**y5_1 = [50, 55, 60, 65, 70]**
**y5_2 = [45, 50, 52, 58, 62]**

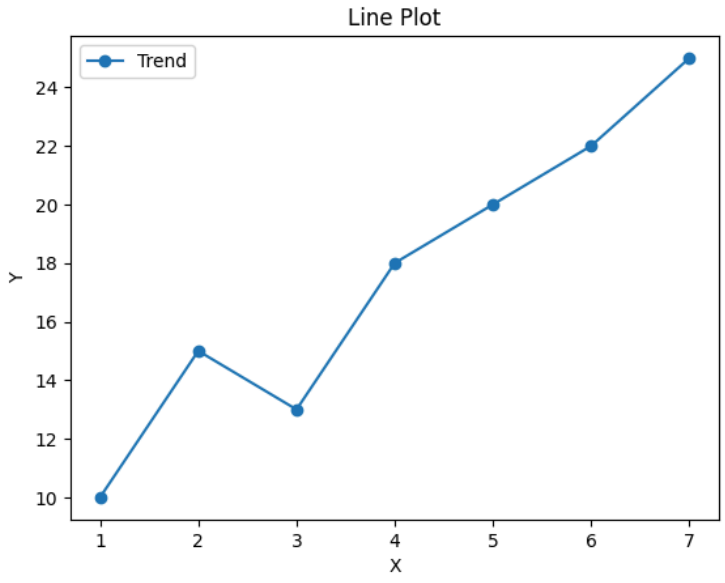
```
import matplotlib.pyplot as plt

x1 = [1, 2, 3, 4, 5, 6, 7]
y1 = [10, 15, 13, 18, 20, 22, 25]
x2 = ['A', 'B', 'C', 'D', 'E']
y2 = [25, 40, 30, 35, 20]
x3 = [5, 10, 15, 20, 25, 30, 35]
y3 = [12, 18, 20, 28, 30, 33, 40]
labels4 = ['Rent', 'Food', 'Transport', 'Savings']
sizes4 = [40, 25, 15, 20]
x5 = [1, 2, 3, 4, 5]
y5_1 = [50, 55, 60, 65, 70]
y5_2 = [45, 50, 52, 58, 62]

plt.figure()
plt.plot(x1, y1, marker='o', label='Trend')
plt.title('Line Plot')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.show()
```

**OUTPUT:**

<Figure size 640x480 with 1 Axes>

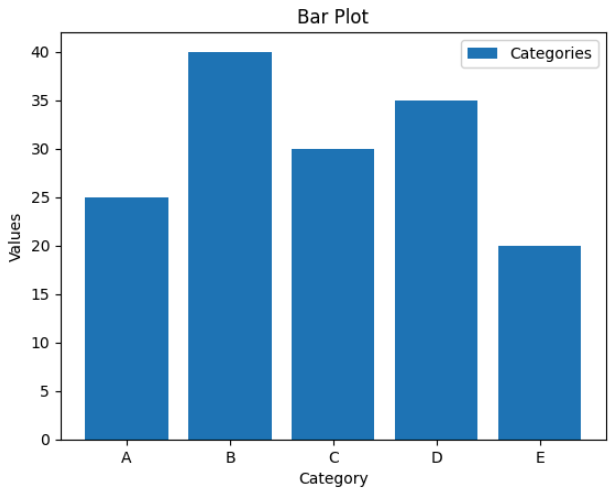



```
plt.figure()
plt.bar(x2, y2, label='Categories')
plt.title('Bar Plot')
plt.xlabel('Category')
plt.ylabel('Values')
```

```
plt.legend()
plt.show()
```

**OUTPUT:**

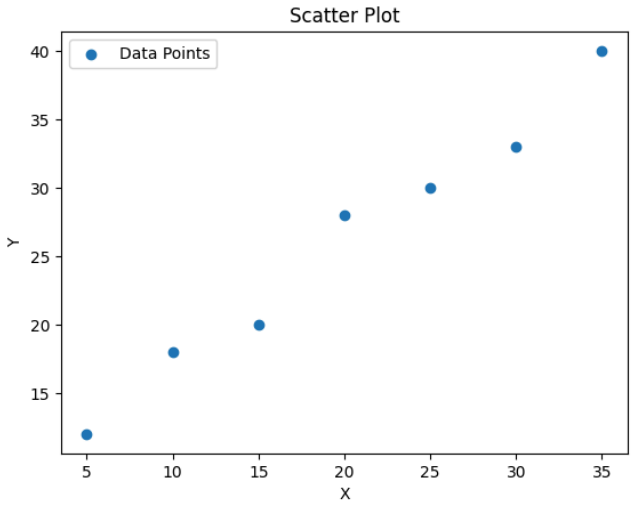
<Figure size 640x480 with 1 Axes>




```
plt.figure()
plt.scatter(x3, y3, label='Data Points')
plt.title('Scatter Plot')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.show()
```

**OUTPUT:**

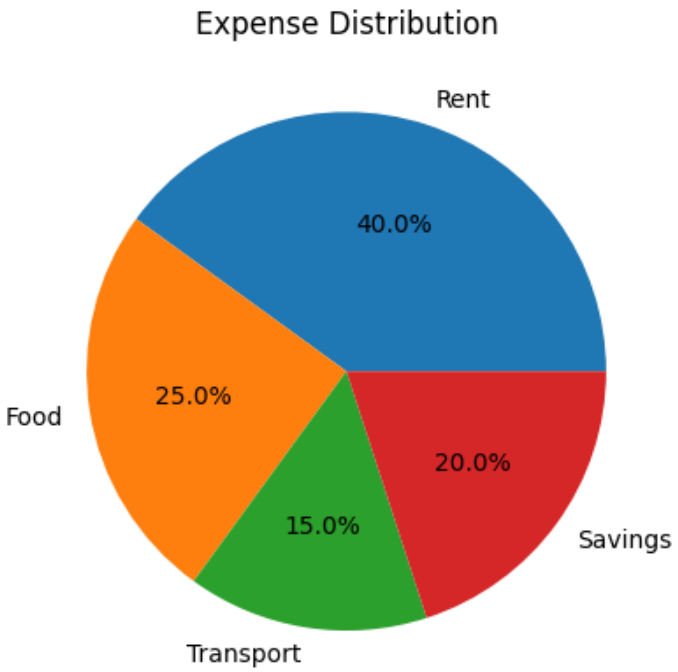
<Figure size 640x480 with 1 Axes>



```
plt.figure()
plt.pie(sizes4, labels=labels4, autopct='%1.1f%%')
plt.title('Expense Distribution')
plt.show()
```

**OUTPUT:**

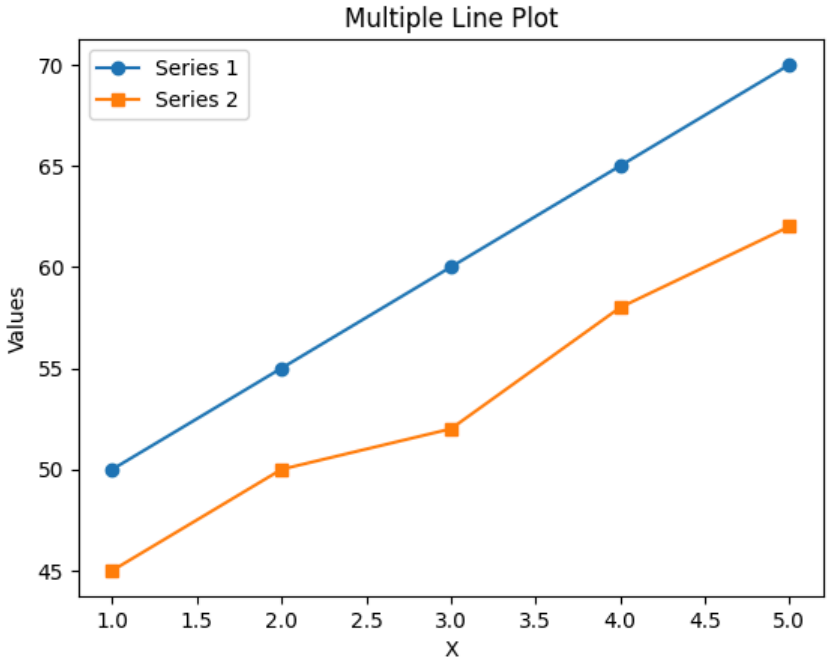
<Figure size 640x480 with 1 Axes>




```
plt.figure()
plt.plot(x5, y5_1, marker='o', label='Series 1')
plt.plot(x5, y5_2, marker='s', label='Series 2')
plt.title('Multiple Line Plot')
plt.xlabel('X')
plt.ylabel('Values')
plt.legend()
plt.show()
```

**OUTPUT:**

<Figure size 640x480 with 1 Axes>




**RESULT:**

Thus, data visualization using line, bar, scatter, pie, donut, and histogram charts was successfully implemented and executed.

# EXPERIMENT 13: PROBABILITY DISTRIBUTIONS AND VISUALIZATION

**AIM:**
To generate, simulate, analyze, and visualize various discrete and continuous probability distributions using Python.

**PROCEDURE:**

- Open Python IDLE and create a new Python program using File → New File.
- Import NumPy and Matplotlib, and import the required statistical functions from SciPy where necessary.
- Generate random samples from normal and standard normal distributions and calculate probabilities and Z-scores.
- Generate samples from binomial and Poisson distributions and analyze the frequency of outcomes and arrival probabilities.
- Use uniform random variables to estimate the value of Pi through simulation.
- Generate samples from exponential and gamma distributions to estimate average waiting times.
- Generate samples from chi-square and Student's t distributions and compare their statistical properties.
- Generate samples from the beta distribution and calculate the sample mean.
- Execute each program by selecting Run → Run Module or pressing F5, and observe the results in the Python Shell and graphical output windows where applicable.
- Record the program code and corresponding outputs in the lab record.

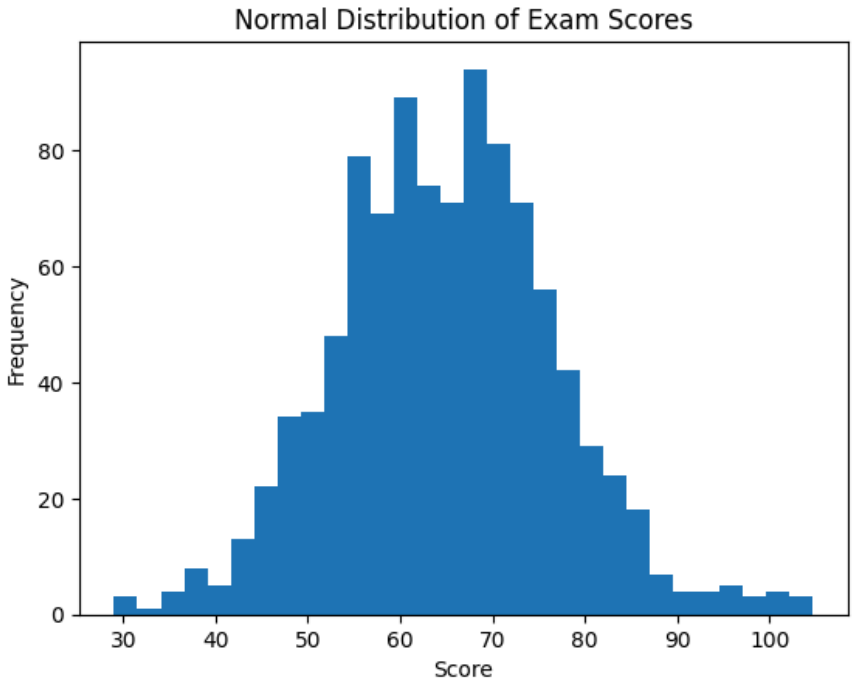
## PROGRAM 1: WRITE A PYTHON PROGRAM TO SIMULATE A NORMAL DISTRIBUTION OF EXAM SCORES AND CALCULATE PROBABILITIES

```
import numpy as np
import matplotlib.pyplot as plt
data = np.random.normal(65, 12, 1000)
prob = np.mean(data > 80)
```

```
print("Probability of score >80:", prob)
plt.hist(data, bins=30)
plt.show()
```

**OUTPUT:**

Probability of score >80: 0.092




**<u>PROGRAM 2:</u> WRITE A PYTHON PROGRAM TO COMPUTE Z-SCORES FOR A STANDARD NORMAL DISTRIBUTION**

```
data = np.random.normal(50, 10, 1000)
z = (data - np.mean(data)) / np.std(data)
print("Mean of Z-scores:", np.mean(z))
print("Std of Z-scores:", np.std(z))
```

**OUTPUT:**

Mean of Z-scores: 3.0553337637684306e-16

Std of Z-scores: 1.0

**PROGRAM 3: WRITE A PYTHON PROGRAM TO SIMULATE A BINOMIAL DISTRIBUTION AND FIND THE MOST FREQUENT OUTCOME**

```
data = np.random.binomial(10, 0.5, 1000)
values, counts = np.unique(data, return_counts=True)
mode_heads = values[np.argmax(counts)]
print("Most frequent number of heads:", mode_heads)
```

**OUTPUT:**

Most frequent number of heads: 5

**PROGRAM 4: WRITE A PYTHON PROGRAM TO SIMULATE A POISSON DISTRIBUTION AND COMPUTE ARRIVAL PROBABILITIES**

```
data = np.random.poisson(4, 1000)
prob = np.mean(data > 6)
print("Probability of >6 arrivals:", prob)
```

**OUTPUT:**

Probability of >6 arrivals: 0.104

**PROGRAM 5: WRITE A PYTHON PROGRAM TO ESTIMATE PI USING UNIFORM RANDOM VARIABLES**

```
x = np.random.uniform(-1, 1, 10000)
y = np.random.uniform(-1, 1, 10000)
inside = x**2 + y**2 <= 1
pi_estimate = 4 * np.mean(inside)
print("Estimated Pi:", pi_estimate)
```

**OUTPUT:**

Estimated Pi: 3.14

**PROGRAM 6: WRITE A PYTHON PROGRAM TO SIMULATE AN EXPONENTIAL DISTRIBUTION AND COMPUTE MEAN WAITING TIME**

```
data = np.random.exponential(scale=2, size=1000)
print("Simulated Mean Waiting Time:", np.mean(data))
```

**OUTPUT:**

Simulated Mean Waiting Time: 2.083760530530420971

**PROGRAM 7: WRITE A PYTHON PROGRAM TO SIMULATE A GAMMA DISTRIBUTION AND COMPUTE AVERAGE WAITING TIME**

```
data = np.random.gamma(shape=3, scale=2, size=1000)
print("Average Waiting Time for 3rd Arrival:", np.mean(data))
```

**OUTPUT:**

Average Waiting Time for 3rd Arrival: approximately 6.270628103078408

**PROGRAM 8: WRITE A PYTHON PROGRAM TO SIMULATE A CHI-SQUARE DISTRIBUTION AND CALCULATE SAMPLE MEAN**

```
data = np.random.chisquare(df=4, size=1000)
print("Sample Mean:", np.mean(data))
```

**OUTPUT:**

Sample Mean: 3.9776842984012433

**PROGRAM 9: WRITE A PYTHON PROGRAM TO COMPARE PROBABILITIES BETWEEN STUDENT'S T AND NORMAL DISTRIBUTIONS**

```
from scipy.stats import t

data_t = t.rvs(df=10, size=1000)
data_n = np.random.normal(0, 1, 1000)
prob_t = np.mean(np.abs(data_t) > 2)
prob_n = np.mean(np.abs(data_n) > 2)
print("P(|T|>2):", prob_t)
print("P(|N|>2):", prob_n)
```

**OUTPUT:**

P(|T|>2): 0.088

P(|N|>2): 0.04

**<u>PROGRAM 10:</u> WRITE A PYTHON PROGRAM TO SIMULATE A BETA DISTRIBUTION AND CALCULATE EXPECTED PROBABILITY**

```
data = np.random.beta(2, 5, 1000)
print("Expected Probability:", np.mean(data))
```

**OUTPUT:**

Expected Probability: 0.2829397617424493

**RESULT:**

Thus, various continuous and discrete probability distributions were successfully generated, analyzed, and visualized.